Medical Radiation Physics,
Lund University,
Sweden

October 20, 2026
LuRaCs version: 0.6.1

LuRaCs - Reference Manual

Erik Ewald
Christian Bernhardsson





LUND
UNIVERSITY



Contents

1 Getting Started **3**
1.1 Introduction . . . 3
1.1.1 LuRaCs-H3 . . . 4
1.1.2 LuRaCs-Si . . . 5
1.2 Installation . . . 5
1.2.1 With `pip` . . . 5
1.2.2 From source . . . 6
1.3 Connecting Detector . . . 6
1.4 Compatible file formats . . . 7
1.5 Package dependencies and Resources . . . 7
1.6 Terms & Object Definitions . . . 8
1.6.1 Abbreviations . . . 10

2 Interface **11**
2.1 Primary Tabs . . . 11
2.1.1 Spectrum . . . 11
2.1.2 Spectrogram . . . 14
2.1.3 Map . . . 17
2.1.4 Measure . . . 18
2.2 Secondary Tabs . . . 20
2.2.1 Spectrum Info . . . 20
2.2.2 Spectrum ROIs . . . 22
2.2.3 Spectrogram ROIs . . . 22
2.2.4 Detectors . . . 23
2.2.5 Isotopics . . . 26
2.2.6 Console . . . 27
2.2.7 System Log . . . 27
2.3 Gamma Tools . . . 27
2.3.1 Calibration . . . 27
2.3.2 Deconvolution . . . 28
2.3.3 Efficiency . . . 29
2.3.4 Peak Features . . . 29
2.3.5 Resolution . . . 30
2.4 Detector Connection & Control . . . 31
2.5 Data Library . . . 32
2.6 Settings . . . 36
2.7 Command Line Arguments . . . 38
2.8 Command Line Interface (CLI) . . . 39

3 Examples **43**
3.1 Quick Starting Several Radiacode Detectors for mobile measurements . . . 43
3.2 Connecting Detective X using direct ethernet . . . 43
3.3 Preparing and Running the LU mobile detector setup . . . 45

4 Internal Function **46**
4.1 Code Structure . . . 46
4.2 Wrapping Detectors . . . 48
4.3 Algorithms . . . 49
4.3.1 Peak Fitting . . . 49

4.3.2 Peak Search 49
4.3.3 Deconvolution 49
4.4 File Types 49
4.4.1 Spectrum, ROI and Instrument (.xml) 49
4.4.2 Spectrogram (.db) 54
4.4.3 Detector Response Matrix (.npz) 55
4.4.4 Simple map tracks (.geojson) 55
4.5 Embedded map 56

A LuRaCs-Xe **59**

B Change Log **60**

1 Getting Started

Licence, Warranty and Research Use

This software is free and open-source software distributed under the terms of the GNU General Public License, version 3 (GPLv3).

The software is provided "AS IS" and WITHOUT WARRANTY OF ANY KIND, to the maximum extent permitted by applicable law. The warranty disclaimer and limitation of liability contained in Sections 15 and 16 of the GPLv3 apply to the software.

Lund University and its contributors make no representation or warranty that the software is accurate, reliable, error-free, uninterrupted, or suitable for any particular purpose.

Research Use

This software is intended as a research and experimental tool for gamma spectroscopy and mobile radiation measurements. It is not intended to constitute a certified measurement system. Measurement results should not be assumed to be accurate or suitable for regulatory, safety-critical, medical, environmental, or other consequential applications without appropriate independent validation.

Measurement results may be affected by the detector, electronics, calibration, measurement geometry, environmental conditions, acquisition parameters, data processing, and other factors.

Users are responsible for evaluating and validating the software and its results for their particular research application.

To the maximum extent permitted by applicable law, Lund University and its contributors accept no liability for any loss, damage, decision, action, or omission arising from or in connection with the use of the software or reliance on measurements or results obtained using the software.

Nothing in this notice excludes or limits any liability that cannot legally be excluded or limited under applicable law.

The complete GNU General Public License version 3 is provided with the software.

1.1 Introduction

The Lund Radiation Analysis Computer Software (LuRaCs) is developed and maintained by the Department of Medical Radiation Physics at Lund University, Sweden. Its development originated from a project benchmarking citizen science radiation detectors, during which the need for a more standardised and versatile software platform became apparent. In particular, integrating data acquisition from multiple detectors, simulations, and subsequent analysis within a single application was identified as a means of simplifying experimental workflows and improving the consistency of future studies.

Developing and maintaining the software in-house also provides a foundation for implementing and evaluating advanced experimental methods, such as spectrum deconvolution. Furthermore, the existing software used for mobile gamma-ray spectroscopy at the department was no longer actively maintained and had become increasingly outdated. Given the importance of reliable radiation protection and emergency preparedness, the ability to operate independently of an internet connection was established as an additional

design requirement. These needs were consequently incorporated into a single software development project.

LuRaCs is therefore primarily intended as a research platform, while also seeking to support the growing field of citizen science. It is designed to accommodate a broad range of users and applications, balancing flexibility and analytical capabilities with ease of use. By integrating data acquisition and analysis in a single, standalone application, LuRaCs aims to provide a common and extensible platform for radiation measurement and research.

In addition to its research applications, LuRaCs is designed with radiation protection, defence, and emergency preparedness requirements in mind. The software is cross-platform and can be deployed across supported operating systems. Its mapping functionality supports offline maps and generic GPS devices, enabling location-based measurements without requiring an internet connection. Furthermore, its use of open file formats, including an interoperable spectral data format, facilitates data exchange with other software and supports integration into existing workflows.

This manual serves as both a user guide and a technical reference, documenting the use of LuRaCs as well as its internal structure and implementation. As an open-source project licenced under GPL-3.0, LuRaCs encourages reuse, adaptation, and further development by researchers who may find the software useful in their own work. Its architecture is designed with extensibility in mind, allowing additional analysis tools and detector integrations to be incorporated with minimal modifications to the existing codebase. In this way, the project aims to facilitate not only the use of the software in a broad range of research applications, but also its continued development and adaptation to emerging experimental needs.

If you use LuRaCs in your research or other projects, please cite the following publication when reporting or sharing your work:

[Full reference to the accompanying publication]

We thank the people who have worked on and contributed to this project.

Erik Ewald, M.Sc.	Software Lead
Christian Bernhardsson, Ph.D.	Project Supervisor
Malte Axner	Contributor
Sigurður Emil Pálsson, Ph.D.	Software Tester

For bugs and other issues with the software, please report this through *Issues* on the project GitHub page:

`https://github.com/EBELU/LuRaCs/issues`

1.1.1 LuRaCs-H3

LuRaCs-H3 is a light weight version of the compiled program indented for systems with limited storage capacity. It excludes large components to reduce the executable file size. This version is approximately 250 MB smaller than the regular version. The following component is excluded in the H3 version.

- The embedded map (position data can still be recorded)

1.1.2 LuRaCs-Si

LuRaCs-Si is a special version of LuRaCs that starts the application in *Si*-mode by default. This is intended for users who do not need and might be confused by the more advanced features. Therefore, it soft locks the application the *Measure*-tab, see section 2.1.4, and hides most other tooling. *Si*-mode can still be turned of and the application will then be identical to the *H3* version.

1.2 Installation

The binary distribution of LuRaCs does not require any installation before use, except for usb drivers on Windows.

Before installation, ensure that Python version 3.10, 3.11, 3.12 or 3.13 is installed on the system. The Python installation can be verified by opening *cmd* on windows or a Terminal on MacOS and Linux and entering `python -version` or `python3 -version`. If Python is correctly installed the version will be displayed.

1.2.1 With `pip`

With Python correctly set up, make a new folder for the installation and open *cmd* or a terminal in that folder. On Windows enter the following commands in *cmd*:

```
# Make a new python environment
python -m venv .venv

# Activate the environment
.\.venv\Scripts\activate

# Install
pip install luracs

# Run
LuRaCs
```

On Linux and MacOS run the following bash commands in a Terminal:

```
# Make a new python environment
python3 -m venv .venv

# Activate the environment
source .venv/bin/activate

# Install
pip install luracs

# Run
LuRaCs
```

This will install all dependencies required for the full version of the application and provide the start script `LuRaCs`.

1.2.2 From source

LuRaCs can be installed from source by cloning the GitHub repository. Installing from source requires a C/C++-compiler for compiling *Cython* extensions. When installing from source to edit the code, run `pip install -e .[dev]` to install in editable mode. This will also install the development tools like `pyinstaller` and `pytest`.

```
# Clone the the GitHub repo
git clone --recursive https://github.com/EBELU/LuRaCs.git
cd LuRaCs

# Install with pip
pip install .
# Run
LuRaCs
```

1.3 Connecting Detector

Table 1.1: Currently implemented devices

Model	Manufacturer	Connections	Note
RadiaCode-1xx	Radiacode Ltd.	USB, BLE	
Raysid	In-View	BLE	
digiBase	Oretec Ametek	USB	Requires Ortec Firmware
digiDart	Oretec Ametek	USB	
DetectiveX	Oretec Ametek	NETWORK	
Identifinder R400	FLIR	NETWORK	

In order to use USB connected devices on Linux the udev-rules must be updated by adding the following permissions depending on the device.

```
# Radiacode
SUBSYSTEM=="usb", ATTR{idVendor}=="0483", ATTR{idProduct}=="f123", TAG+="uaccess",
↪  MODE="0660"

# digiBase/digiDart
SUBSYSTEM=="usb", ATTRS{idVendor}=="0a2d", GROUP="users", MODE="0666"
```

Connecting devices over the network typically requires instrument-specific configuration, so no general guide is provided. See example 3.2 on how this can be done for the Ortec Ametec Detective X.

Connecting a digiBase requires the files containing the necessary firmware. These files can be found in the installation directories of the `Maestro` and `Gamma Vision` software provided by Ortec. LuRaCs does not provide these files. Users are responsible for acquiring these files in accordance with applicable licences. By importing these files to the driver library they can be reached by LuRaCs. In LuRaCs, select `File -> Import`

`Drivers` and select the files. If they appear in `File -> Drivers` they can now be reached by LuRaCs.

1.4 Compatible file formats

LuRaCs is compatible with a selection of file formats, both standard and other third party formats. However, LuRaCs is not primarily intended to support general spectrum analysis. The user may consider other analysis applications for such purposes, e.g. InterSpec[1] instead.

Table 1.2: Compatible spectrum formats.

Source	Extension	Embedded data
Radiacode	.xml	Counts, background, live time, real time, calibration
Several	.n42	Counts, background, live time, real time, calibration
Ortec	.spe	Counts, live time, real time, calibration
Canberra	.tka	Counts, live time, real time

Table 1.3: Compatible track formats.

Source	Extension	Embedded data
Radiacode	.rctrk	Geolocation for Count- & Dose Rates

1.5 Package dependencies and Resources

We are grateful to the develops and maintainers of the packages used in the application. Both the projects that are included as dependencies and those that we forked and modified. All licences are collected in the `/licences` directory in the GitHub repo and packaged with compiled versions as well. Forked packages have their original licences and notices retained as they were in the original code, with our changed noted.

[1] *Sandialabs/InterSpec*. Sandia National Laboratories, Aug. 28, 2025. URL: `https://github.com/sandialabs/InterSpec` (visited on 08/29/2025).

Table 1.4: Dependencies, their licenses, and source repositories.

Package	License
Bleak	MIT
NumPy	BSD-3-Clause
PyQtGraph	MIT
PySide6	GPL-3.0 (Chosen option)
openpyxl	MIT
PyUSB	BSD-3-Clause
lxml	BSD-3-Clause
FastAPI	MIT
Uvicorn	BSD-3-Clause
Requests	Apache-2.0
plotext	MIT
prompt_toolkit	BSD-3-Clause
libusb_package	Apache-2.0
uncertainties	BSD-3-Clause
Markdown	BSD-3-Clause

Table 1.5: Forked projects that were used in the application.

Package	License
MapLibre GL JS	BSD-3-Clause
pmtiles	BSD-3-Clause
readiacode	MIT

Table 1.6: Other third party resources used.

Resource	License
Meterial Design Icons	Apache 2.0
Catpucchin Theme	MIT

1.6 Terms & Object Definitions

Spectrum is a data object that contains count histograms from a radiation detector and associated meta data. A spectrum contains the following information:

- Spectrum name (Ex. 'Radium_Watch_10cm').
- Fore- and Background spectrum.
- Number of channels, fore- and background must have the same number of channels (Ex. 1024).
- Optional remark or description.
- Detector identifier (Ex. RC-110-001234).
- Detector model (Ex. RadiaCode-110).
- ROIs and analysis results for defined ROIs in the spectrum e.g. counts and parameters from a curve fit, see the *ROIs* section bellow.

- Calibration coefficients for converting channel number to energy (Ex. [0.00024, 0.989, 15.4]).
- Calibrated X-axis expressed in keV [0, 1.233, ..., 2997.23, 3000].
- Reference to the instrument used for calculations, see the *Instruments* section bellow.

Region Of Interest (ROI) is a spectrum region bounded by an upper and lower energy. In LuRaCs ROIs can be applied both in a spectrum and in a (recorded/ongoing) spectrogram. When applied to a spectrum a Gaussian function can be fitted to a peak contained inside the ROI, see section 2.1.1

When applied to spectrograms ROIs do not perform any curve fitting, only the summed counts inside the ROI are calculated. The time series produced from a spectrogram ROI can be viewed separately in the map. Spectrograms can use special ROIs called *Calculation ROIs* or *CalcROIs*. The CalcROIs are calculated by adding, subtracting, or dividing the data from two different ROIs. For example, CAL_0 (a Calculation ROI) can show the results of ROI_0 / ROI_1 where ROI_0 is placed over the K-40 peak and ROI_1 is placed over the Cs-137 peak.

Spectrogram is the primary method for recording time series data. It allows for measuring the difference in counts in each channel between time points separated by a fixed time. It also interfaces with a GPS function and can store geolocational information for mobile measurements. A spectrogram is a rather complicated object and is discussed further in other parts of this manual, see section 4.4.2.

LuRaCs provides the following options when creating a spectrogram:

- Detector selection for data collection.
- Name of the new spectrogram.
- Save interval (the time between each row/measurement point).
- Whether channels should be concatenated. For example, concatenating by a factor 2 means summing two adjacent channels into one, thereby halving the total number of channels. This is useful for reducing file size and improving visual analysis.

If the detector used for acquiring the spectrum associated with the spectrogram is energy calibrated, the same calibration is used for the spectrogram.

Instrument s represent parameters of an external *detector* and are divided into *Generic* and *Unique* instruments. *Generic Instrument* represent a detector model, like 'RadiaCode-110' or '3"x3" NaI + digiBase'. All instruments of the same model share some properties, of which the *Generic Instrument* stores the following:

- Model name.
- Manufacturer.
- Detector material.
- Sensitive volume size.
- Energy Resolution.

- Intrinsic Energy Efficiency.

A *Unique Instrument* represents a specific detector of a certain model. The *Generic Instruments* serve as a basis for quickly making a new *Unique Instrument*. A *Unique Instrument* can store the same information as a *Generic Instrument* in addition to an energy calibration and uses a serial number for identification. *Unique Instruments* can be assigned to a spectrum from which certain tools can pull required data for calculations and they can store the energy calibration for instruments that do not store it themselves.

Detector denotes an external instrument for detecting radiation that LuRaCs can sample data from. This is in contrast to an *Instrument* which represent detector parameters and not the detector itself. They can be connected through USB, Bluetooth or a network. Each type of detector have their own 'client'. LuRaCs interacts with detectors through clients which contain the code needed to interact with the physical detector but each client can be implemented differently. However, the end-user interacts with the detectors the same way no matter how the client is coded. This structure makes detector implementations in LuRaCs flexible and easily extensible.

1.6.1 Abbreviations

Table 1.7: List of abbreviations used in this manual

BLE	Bluetooth Low Energy
CLI	Command Line Interface
CPS	Counts Per Second
CsI	Caesium-Iodide scintillator
HPGe	High Purity Germanium semi-conductor
LuRaCs	Lund Radiation analysis Computer software
GPS	Global Positioning System
GUI	Graphical User Interface
ML-EM	Maximum Likelihood - Expectation Maximisation
NaI	Sodium-Iodide scintillator
USB	Universal Serial Bus

2 Interface

2.1 Primary Tabs

2.1.1 Spectrum

The spectrum tab shows spectra as a count histograms, retrieved from a connected detector or loaded from a compatible file. It can show several spectra combined in one plot or in separate tabs within the spectrum tab, see Figure 2.3. This can be changed under `Options -> Tabbed-/Combined Spectrum View` (then selecting the files or data to be shown).

Background spectra can be viewed as: *Background overlay* and *background subtract* or not at all: *No background*. *Background overlay* shows the background spectrum overlaid on top of the main (foreground) spectrum. *Background subtract* subtracts the background, channel by channel, from the foreground spectrum. Note that this is a hard background subtraction and will thus effect spectrum ROI calculations. Both *Background overlay* and *Background subtract* are limited to show count rate values.

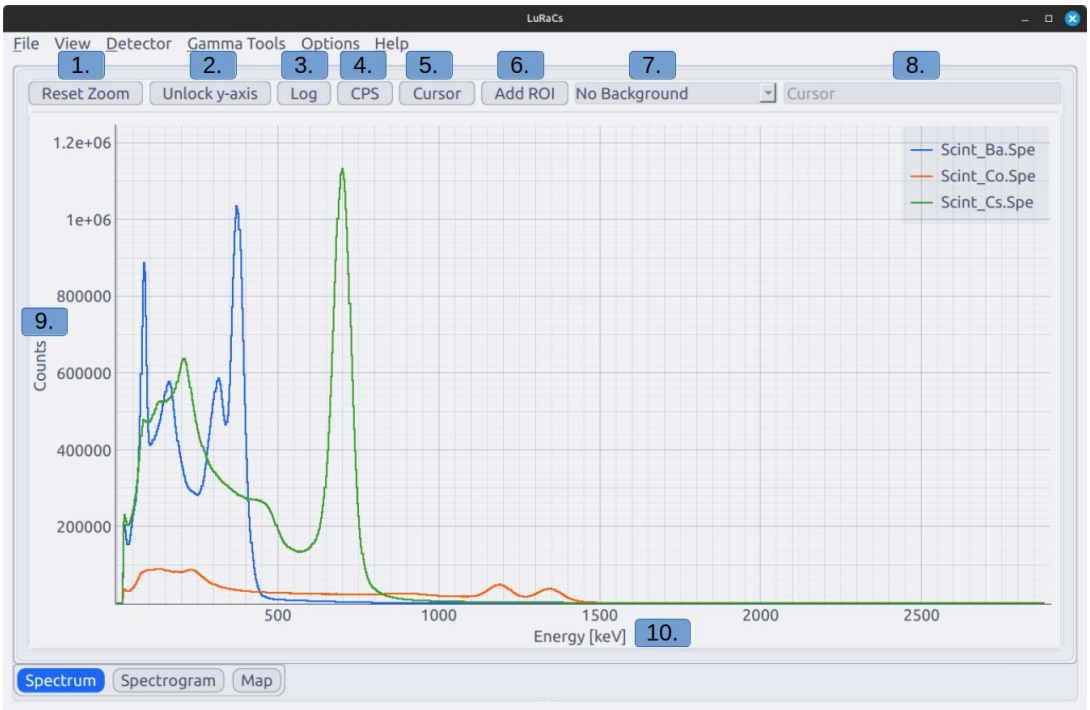


Figure 2.1: Spectra of Ba-133, C-60 and Cs-137 displayed in *Combined Spectrum View*.

1. **Rest Zoom** resets the zoom of the spectrum view and locks the y-axis if unlocked.
2. **Unlock y-axis** allows the y-axis to be zoomed independently (by scrolling the wheel of the mouse) and disables autoscaling.
3. **Log** set the scale of the y-axis to logarithmic or reset back to linear scale.
4. **CPS** normalises the spectrum to the live time, or real time if live time is not available but real time is.
5. **Cursor** toggles the spectrum cursor, see Figure 2.3.
6. **Add ROI** adds a customisable *region of interest* to the spectrum, see Figure 2.2.
7. Switch the use of background spectrum. Options are *No background*, *Background overlay* and *background subtract*. See the section above.
8. Display for info from the cursor, see Figure 2.3.
9. The y-axis show either total counts or time normalised counts in units of 1/s.
10. The x-axis always display energy in units of keV.

Regions of interest (**ROI**) are sub regions of the spectrum bounded in energy and are useful in the analysis of spectrum peaks. Spectrum peaks can in most cases be approximated with a Gaussian function, knowing the function parameters is useful for many analyses. LuRaCs provides a method for fitting Gaussian functions to peaks inside ROIs and do so by default. These fits are required by other tools in the application. However, ROIs can also be configured to not perform fits. In these cases, only the total counts within each ROI are calculated.

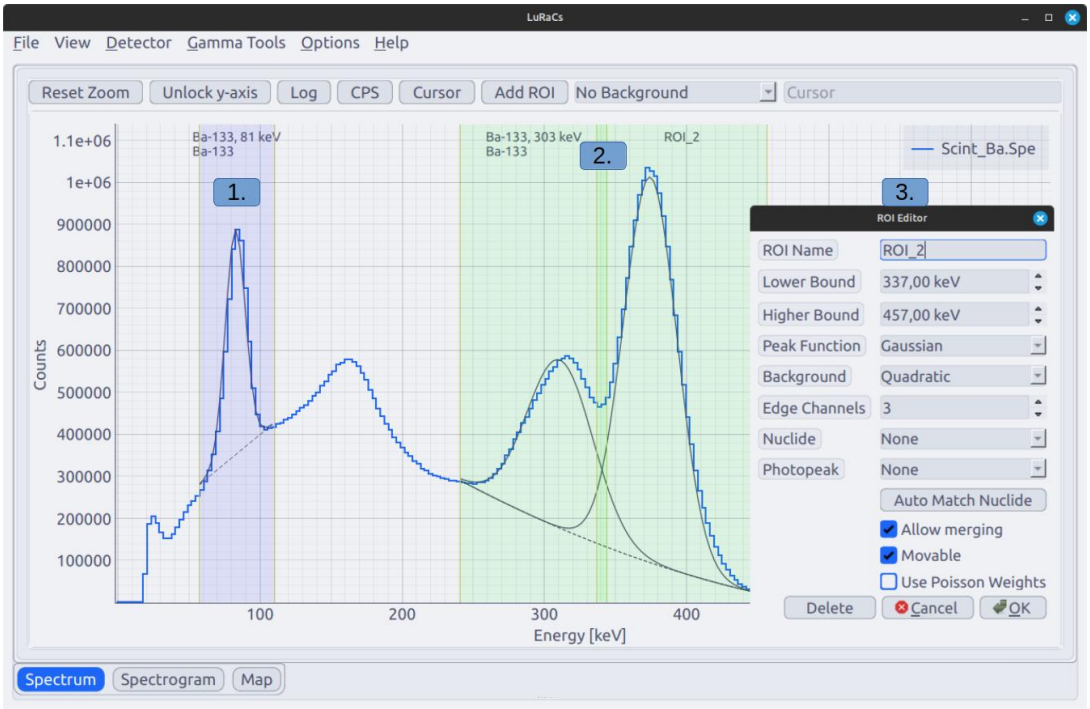


Figure 2.2: A Ba-133 spectrum with three ROIs, two are merged and fitted to the same background. The ROI editor is open for the right most ROI (ROI_2).

1. A ROI with one peak and a fitted Gaussian peak and linear background.
2. Two merged ROIs that fit two Gaussian peaks on a shared quadratic background.
3. The ROI editor for 'ROI_2', see Table 2.1.

Table 2.1: ROI Editor options

Field	**Description**
ROI Name	An alias for the ROI used for display purposes and easier identification.
Lower Bound	The lower energy bound of the ROI.
Upper Bound	The upper energy bound of the ROI.
Peak Function	The type of function fitted to the peak. Options are: *Guassian*, *None*
Background	Type of function used as peak background. Options are: *None*, *Linear*, *Quadratic*
Edge Channels	The number of channels inside the ROI from the upper and lower bounds of the ROI used to fit the background.
Nuclide	Assign a nuclide from the reference library to the ROI. Used for calculations by other tools.
Photopeak	Choose the specific photon emission associated with the peak in the ROI. Options are based on the chosen nuclide.
Auto Match Nuclide	Attempt to automatically find the nuclide and emission of the peak. If any nuclides are checked in the *Isotopics*-tab, see section 2.2.5, the match is limited to these. Blind matches are often unreliable.
Allow Merging	Allow the ROI to merge with other ROIs with merging allowed.
Movable	If the ROI can be moved or its bounds adjusted by using the mouse.
Use Poission Weights	Apply a $1/\sigma$ weight to the optimisation to force the peak to emulate a Poission distribution.

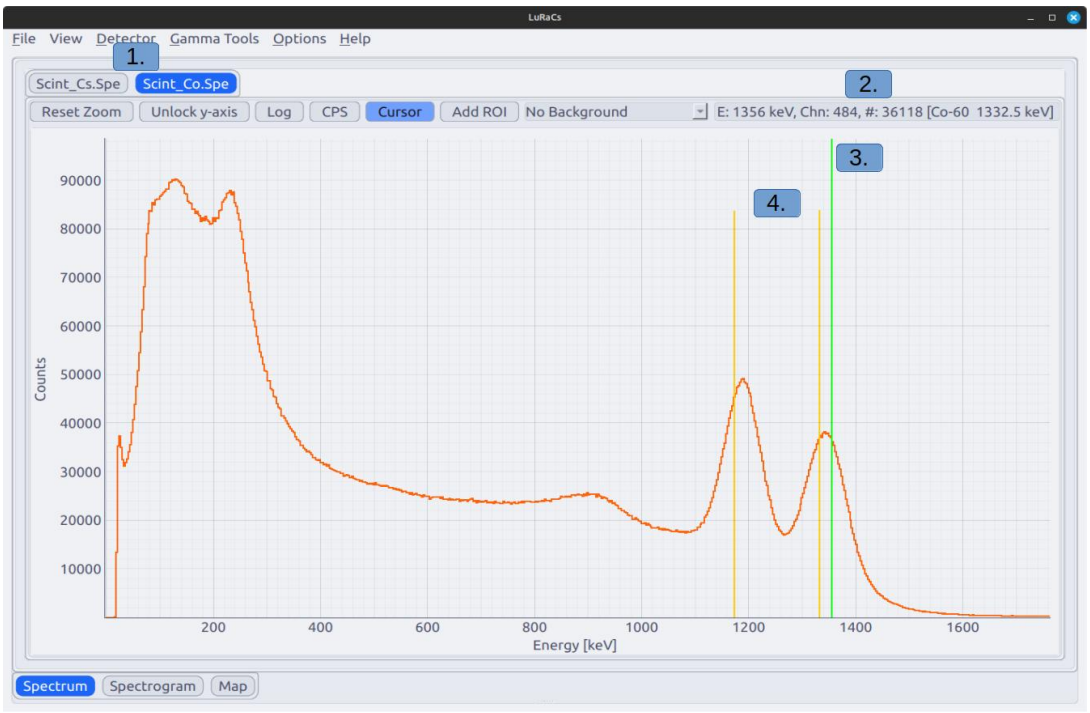


Figure 2.3: Co-60 spectrum with an active cursor.

1. Tabs for selecting spectrum in *Tabbed Spectrum View*.
2. Spectrum information at the position of the cursor, showing of the corresponding energy and channels number, the number of counts in the channel, closest reference emission line as [*nuclide Emission Energy Decay chain if applicable*]. Channel and channel counts are only displayed in tabbed spectrum view.
3. Cursor in the spectrum.
4. These always appear for the emission closest to the cursor in energy and does not weigh intensity or anything else. This can be disabled by `View -> Spectrum -> Show Cursor Emissions`.

2.1.2 Spectrogram

A spectrogram allows acquisition of data time-series. A spectrogram stores data with specified time intervals and can be displayed in a waterfall plot. It is also the function LuRaCs utilise for mobile measurements. If a GPS is connected, the geolocation data is automatically stored in the spectrogram. These measurements can be visualised on the embedded Map (or an online map).



Figure 2.4: Acquisition of a spectrogram with the *Time Selector* active and one ROI.

1. Opens a dialog for starting a new spectrogram.
2. Close the viewed spectrogram.
3. Shortcut to the Data Library Spectrogram page, see section 2.5.
4. Toggle the currently loaded spectrograms. Status is shown to the left as [ACTIVE], [LOADED] or [PAUSED].
5. Pause or Resume data collection, requires that the same detector used to collect the previous data is connected.
6. Options menu, see Figure 2.5
7. Current information stored in the spectrogram and information about the *Time Selector*, if active.
8. Count histogram. Shows either total accumulated spectrum when loading a previously acquired spectrogram, summed channel counts inside the *Time Selector*, if active, or counts in the latest row.
9. ROI illustrated in the histogram, can be removed with Mouse-2.
10. Waterfall diagram of the spectrogram.
11. *Time Selector* to study the summed spectra within a specific time interval. Can be toggled in the options menu (6.).
12. Data break line, illustrates that the time between measured points exceeds the Time Interval. Can indicate a lost connection and thus caution should be taken to verify data integrity. Can be toggled in the options menu (6.).
13. Y-axis displays the timestamps of when each row was measured.
14. X-axis display spectrum energy, if calibrated.
15. Color slider for the waterfall diagram. The color map can be changed by clicking Mouse-2 on the gradient bar.

The spectrogram format is based around an SQLite database, see section 4.4.2 for further details on the format. The following data is stored for each time point in the spectrogram:

- Timestamp.
- Average count rate measured since the last time point.

- Average dose rate measured since the last time point.
- Device temperature at the time the data is stored.
- Longitude at the time the data is stored, if a GPS is available.
- Latitude at the time the data is stored, if a GPS is available.
- Spectrum difference since the previous data point was stored, as compressed binary.
- Meta data, stored as a utf-8 encoded and zlib compressed json-string. Examples of stored metadata include extended GPS data.

Exporting a spectrogram is done trough the Data Library, accessed via the *Load*-button (2.).

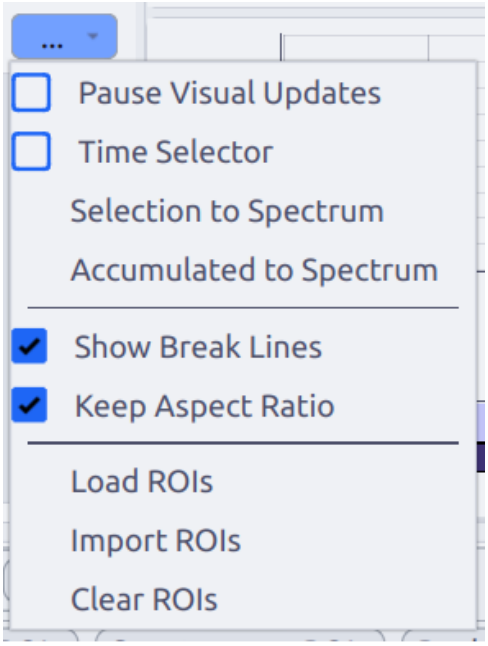


Figure 2.5: Dropdown options menu in spectrogram pane, see Table 2.2.

Table 2.2: Description of the possible operations in the spectrogram options menu

Field	Description
Pause Visual Updates	Stop the view spectrogram from updating while the measurement is still on going in the background. **This does not stop the recording of the spectrogram**.
Time Selector	Add a selected ROI in the spectrogram (11.) that can indicate the specific time interval, as opposed to all other ROIs that have energy bounds, and show the sum over time of the channels in the top count histogram (8.).
Selection to Spectrum	Export the data within a selected time interval to the Data Library as a spectrum.
Accumulated to Spectrum	Export the accumulated counts to the Data Library as a spectrum.
Show Break Lines	If the red lines (12.) indicating if the interval between two rows exceed the specified save interval should be shown.
Keep Aspect Ratio	If the pixels in the spectrogram should always be square. For detectors with many channels it can be useful to turn this off. If this option is off the pixels in the spectrogram may not align with the corresponding bar in the count histogram.
Load ROIs	Shortcut to the ROI tab of the Data Library.
Import ROIs	Load ROIs from a LuRaCs XML-file.
Clear ROIs	Remove all spectrogram ROIs

2.1.3 Map

Loading a map enables viewing of geographic measurements made with a spectrogram and a GPS. Maps can be loaded from online API requiring an internet connection or loaded from a *.pmtiles*-file for offline use. Compatible APIs and files are offered by several sources, the software *QGis* offers support for exporting *.pmtiles* from other formats using plugins.

Disclaimer LuRaCs does not provide or distribute any map data. Users are responsible for ensuring that such data or tile services they use with LuRaCs are obtained and used in accordance with the applicable licenses, terms, and usage restrictions.

The map is rendered using Qt's web engine, making it a powerful but resource demanding tool. Using offline maps can require a lot of system memory since the server needs to cache tiles for a smooth view. It is therefore recommended to use a system with enough memory when this option is applied. However, both the map and datapoints are rendered using GPU acceleration with WebGL, allowing for 10 000s of points to be displayed without excessive stress on the system. For reference, a Raspberry Pi 5 with 8 GB of RAM can render a 1.4 GB map file without any stutter, total memory usage was approximately 2 GB.

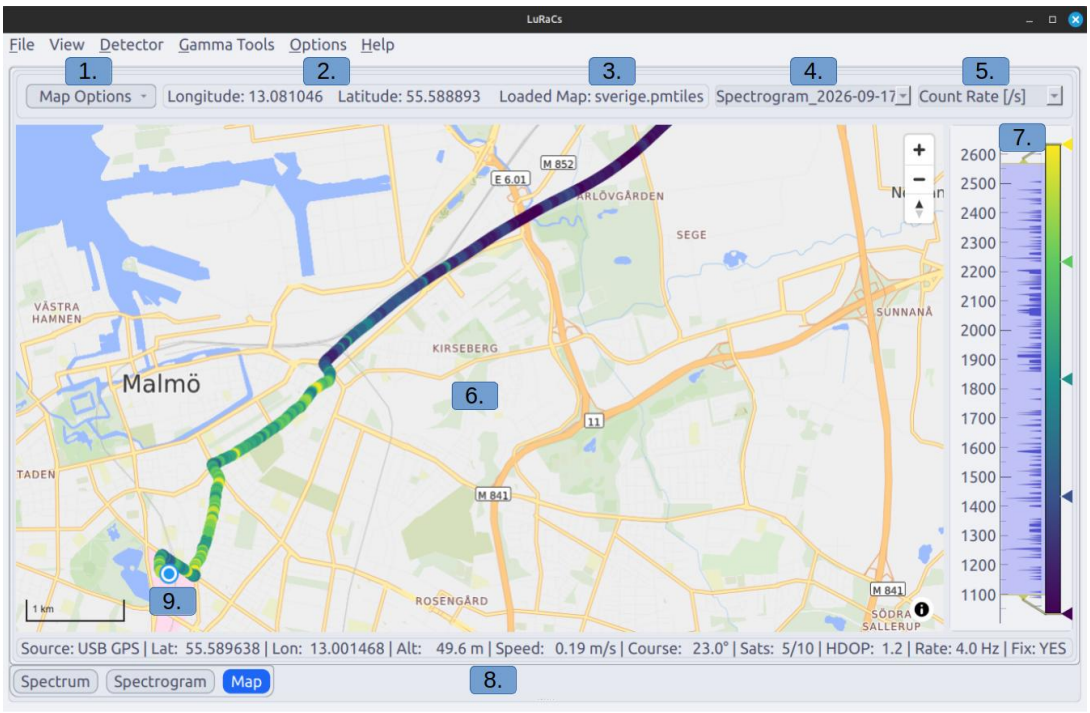


Figure 2.6: Map with a loaded track and a connected GPS.

1. Options menu for loading maps, importing and exporting simple tracks.
2. Longitude and Latitude for the position of the cursor.
3. Name of the loaded map.
4. List of loaded spectrograms. If the selected spectrogram does not contain any geolocation data, no measurement points are displayed.
5. Selector for what data is shown: total count rate, dose rate or count rate inside a loaded ROI.
6. Main map view.
7. Selector for color gradient, used for the track points and a histogram over the measured values. The color map can be changed by clicking Mouse-2 on the gradient bar.
8. Live data from the GPS.
9. Current GPS position, always displayed as a white and blue circle.

Load a map is done from `Map Options -> Load Online/Offline Map`. An offline map takes the form of a *.pmltiles* file. By choosing this option prompts the selection of such a file. Choosing to load an online map prompts the user to enter a URL, to a tile serving API, possibly including an API key. The checkbox allows for saving the most recently used URL. If a personal API key is used, be aware that URLs are saved unencrypted.

2.1.4 Measure

The measure tab allows for making controlled measurements with a simplified interface. These measurement can then be exported as clean, human readable reports while still being valid spectrum files. The interface is designed to be intuitive and this tab serves as the only interface for LuRaCs-Si, as described in the introduction. Buttons will indicate the order they should be pressed and the only submenus required are to connect detectors and to export the report.

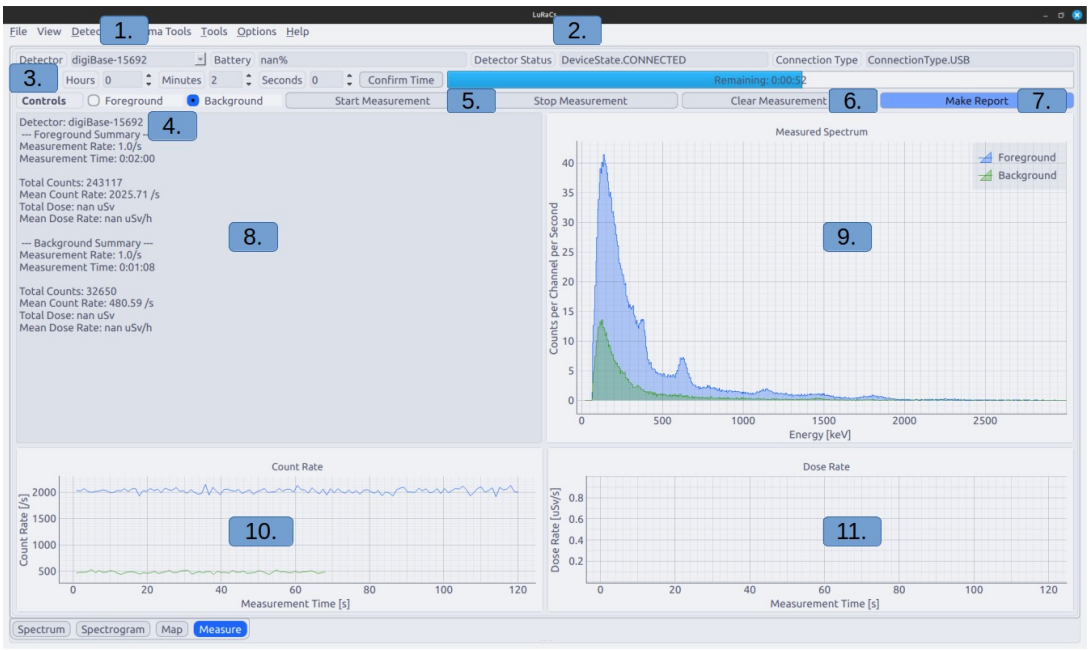


Figure 2.7: The measure view with an on going measurement using a 3"x3" NaI detector. The foreground spectrum is of an antique containing Ra-226 and the background is a regular office. The detector does not have a battery and does not output dose data, therefore these reading are `nan`.

1. Selected detector. If the detector is changed all measurements are reset to avoid mixing measurements for different detectors. This is preceded by a question dialog so results are not lost by accident.
2. Detector status bars, shows the same data as in the *Detectors*-tab, see section 2.2.4
3. The measurement timer to determine the length of a measurement. The duration is set in the spinboxes and must be applied with `Confirm Time`, which while be marked upon changing a time value. The bar shows remaining time. The same time can be used for fore- and background without applying it again.
4. Selects if the measurement is the fore- or background.
5. `Start Measurement` and `Stop Measurement` do what they say. Each time an ongoing measurement is stopped the timestamp is logged and included in the report.
6. Clears the measurement selected in (4.).
7. Opens the Report Dialog for creating a report form the measurement, see Figure 2.8.
8. Main information window that displays summary information about the measurements.
9. Count histogram of the fore- and background. Pulls the energy calibration form the spectrum in the *Spectrum*-tab.
10. Count Rate graph.
11. Dose Rate graph, always in µSv/h

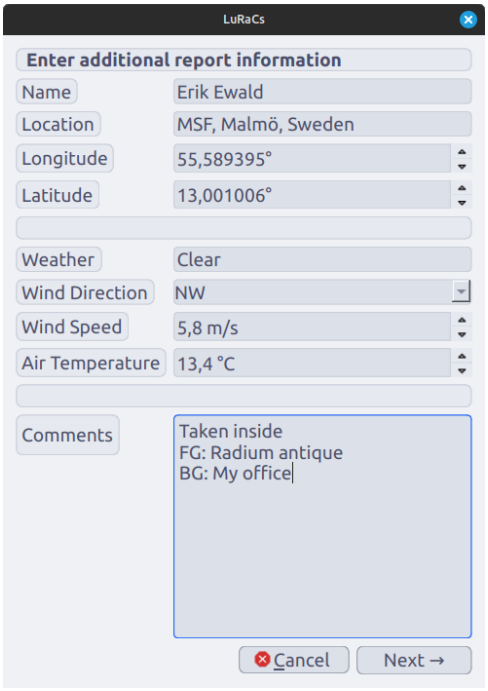


Figure 2.8: Report Form opened by pressing (7.) in Figure 2.7.

2.2 Secondary Tabs

2.2.1 Spectrum Info

Displays information about loaded spectra, both from files (which ones, write out) and detectors memory.



Figure 2.9: Spectrum Info Tab. For imported spectra with other formats than `XML` or `N42` file extensions are kept in the name.

Table 2.3: Description of the columns in the Spectrum Info tab

Column	Description
Column 0	Color button allowing for changing the color of the displayed. Options button marked with '...', see Table 2.4.
Spectrum	The name of the spectrum or connected detector. A spectrum retrieved from a detector is automatically given the same name as the detector, this can not be changed unless the spectrum is saved or exported.
Type	*Foreground* or *Background*, these share the same name in the 'Spectrum'-column for the same spectrum. This division exists because a spectrum can contain both foreground and background that have different measurement times and total counts.
Live Time	The live time (h:min:s) of the measurement, accounting for dead time.
Real Time	The real time (h:min:s) of the measurement.
Calibrated	*True* or *False*, if an energy calibration is detected and applied.
Instrument	Which instrument is assigned to the spectrum, see section 1.6

Table 2.4: Description of the options menu opened by the button marked '...' on each row. The type 'Background's are limited to only the 'Remove' option.

Field	Description
Save	Save the spectrum to the Data Library.
Export	Export the spectrum to a standalone file in the following formats: `XML`, `CSV`, `xlsx`.
Hide/Show	Toggle visibility of the spectrum in the spectrum view. ROIs are not applied to hidden spectra.
Edit	Edit properties of the spectrum, see **??**.
Remove Spectrum	Remove the spectrum and disconnect a detector associated with the spectrum.

2.2.2 Spectrum ROIs



Figure 2.10: Spectrum ROIs tab

1. Remove all loaded ROIs from all spectra.
2. Debug view of all data contained in the loaded ROIs.
3. Apply time normalisation to the ROI data.
4. Export the acquired and calculated data, including live time, of each ROI to a CSV-file.
5. Save ROI references to the Data Library, see section 2.5, for later access. These ROI references save the settings of the ROI (bounds, emission, name, fit type, etc.) but not the ROI data.
6. Icon to display if the spectrum is background subtracted. Background subtraction effects the results of the ROI calculations.

2.2.3 Spectrogram ROIs

Spectrum ROIs apply a curve fitting whereas Spectrogram ROIs do not. When applying a ROI to a spectrogram it sums the counts from all the channels within its bounds and do so for every row in the spectrogram. The results are displayed in count rate, as the number of counts are normalised to the save interval. This makes it easier to compare spectrograms recorded with different save intervals.



Figure 2.11: Spectrogram ROI tab showing two loaded spectrograms with two loaded ROIs. A *Calculation ROI* is also added.

1. Plots of two spectrogram ROIs evaluated on two separate spectrograms.
2. Calculation ROI
3. Button for removing the calculation ROI
4. Settings for the first spectrogram in the calculation ROI. The first combobox lists loaded spectrograms and the second selects the data, total count rate or count rate from a ROI.
5. Mathematical operations for comparing the ROI values are: `+`, `-` or `/`.
6. Settings for the second spectrogram in the calculation ROI. These setting are the same as for the first (4.).
7. ROI names. Calculation ROIs currently only have a tag containing a serial number on the form `CAL_X`. Currently, they can not be renamed.
8. ROI data in text form. Shown is the latest value and (mean+-std) calculated from the data shown in the plot.

During mobile measurements LuRaCs offers the selection to monitor several ROIs simultaneously. For this e a 'Compact View' can be used. This is enabled under `View -> Spectrogram ROis -> Compact View`. This function strips all GUI elements from the plots, except the y-ticks and the plot itself.

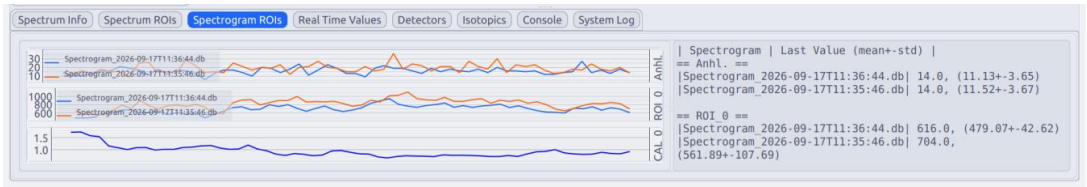


Figure 2.12: Same spectrograms and ROIs as in Figure 2.11 but in compact view.

Calculation ROIs are plots calculated from the results of other ROIs. **These perform element wise operations and do not account for differences in measurement parameters**. Element 0 from ROI 0 is combined with Element 0 of ROI 1, regardless of measurement date or spectrogram save interval.

2.2.4 Detectors

Information and status of connected detectors are displayed in this tab.

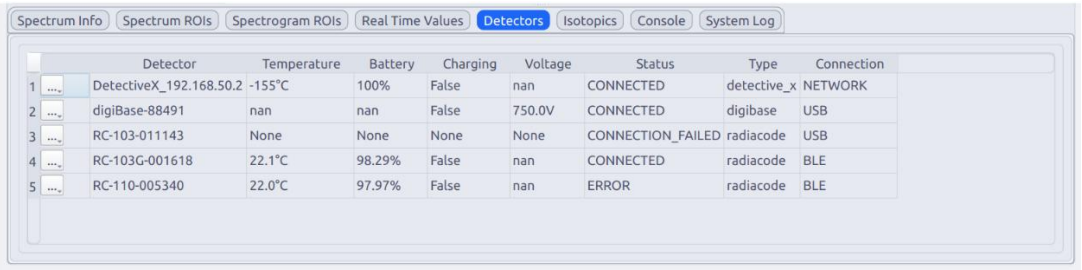


Figure 2.13: List of connected detectors. An entry can not be removed, if a detector is disconnected it is listed as `STOPPED`.

Table 2.5: Description of the columns in the Detectors tab

Column	Description
Column 0	Options button marked with '...', see Table 2.6.
Detector	The name of the detector.
Temperature	The temperature reading from the detector in °C. Most detectors report the ambient temperature but some, like the Detective X, report the cooled crystal temperature, as seen in Figure 2.13.
Battery	The current battery level in %.
Charging	Indicate if the detector is connected to a power source.
Voltage	The actual voltage reported of the detector in V.
Status	Detector state reported as one of the following options: • `UNINITIALIZED` - Detector client created but not started. • `CONNECTING` - The detector is starting. • `CONNECTED` - The detector is connected and running. Default for normal detector function. • `CONNECTION_FAILED` - Detector start failed. Check System Log. • `CONNECTION_LOST` - Connection was lost after a successful start. Not universally implemented. • `STOPPING` - The detector has received and acknowledged the command to stop. • `STOPPED` - The detector has successfully stopped. • `ERROR` - An error occurred while connected and the detector is no longer functioning as expected.
Type	Type of detector by the name defined in the detector client.
Connection	How the detector is connected. Can be `BLE`, `USB` or `NETWORK`.

Table 2.6: Description of the options menu in the detector tab, opened with the button marked '...' in the table.

Field	Description
Disconnect	Sends a stop signal to the detector.
Reset Spectrum	Signals to the selected detector to clear its spectrum.
Start Acquisition	Signals to the selected detector to start spectrum acquisition.
Stop Acquisition	Signals to the selected detector to stop spectrum acquisition.
Make Instrument	Shortcut for creating a *Unique Instrument* for the detector based on a `Generic Instrument` and save the current calibration of the connected spectrum. Useful for detectors that do not Library their own energy calibration.
Settings	Opens the detector settings dialog, see Figure 2.14 and Figure 2.15.

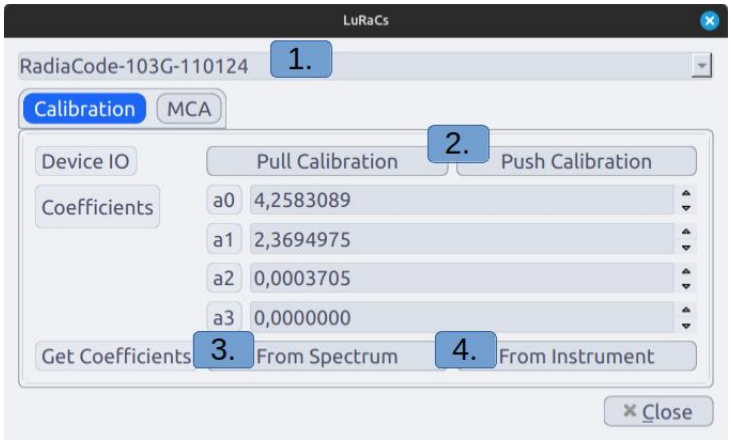


Figure 2.14: Detector settings dialog showing the options for setting device calibration.

1. Unique Instrument from the library to pull settings from.
2. Interact with the detector. `Pull Calibration` will request a calibration from the detector set in the coefficients buffer, not possible for all detectors. `Push Calibration` will attempt to set the calibration of the detector, not possible for all detectors.
3. Pull the calibration coefficients from the spectrum connected to the detector and set in the coefficients buffer.
4. Pull the calibration coefficients from the instrument set in (1.) and set in the coefficients buffer

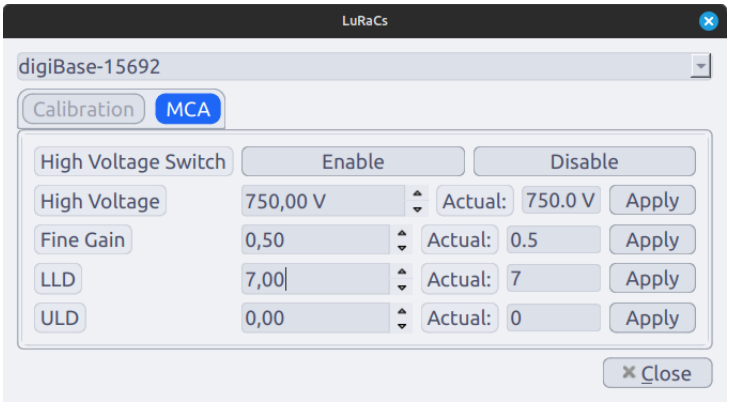


Figure 2.15: Detector settings dialog showing the options for MCA settings. The first column of spinboxes show the requested setting and the second row shows the actual value reported by the instrument. `Apply` attempts to set the given value to the detector.

LLD Lower Level Discriminator
ULD Upper Lever Discriminator

2.2.5 Isotopics

LuRaCs provides a library of data for a selection of common radionuclides that can be accessed under the tab Isotopics. This datacan be used to draw reference lines for emissions in the spectrum plot. Most of the nuclide data is sourced from *Laboratoire National Henri Becquerel*, the others from sources elsewhere. A citation is provided for each nuclide whereas full references can be found under `Help -> Bibliography`.

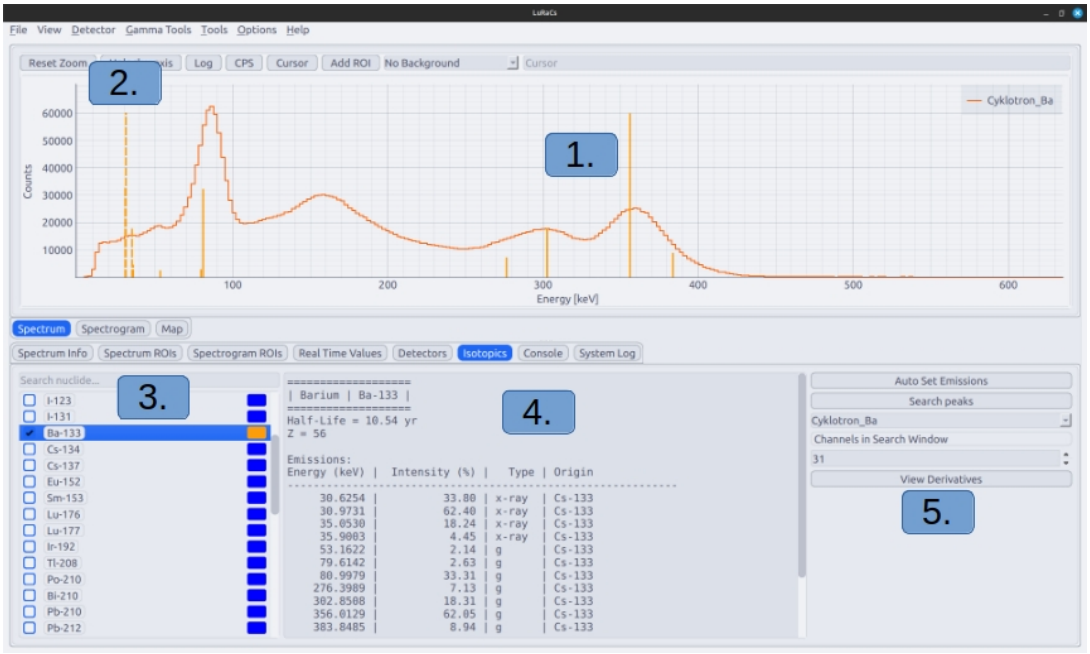


Figure 2.16: Ba-133 spectrum with Ba-133 selected in the Isotopics Tab.

1. Spectrum and gamma emission lines, drawn as solid lines. Line height corresponds to emission intensity,
2. X-ray emission lines, drawn as dashed lines. Line height corresponds to emission intensity.
3. Nuclide list, select nuclide with the checkbox to the left and change colour of the gamma emission lines with the right coloured button.
4. Information of the current marked nuclide.
5. Peak automation, see bellow.

Auto matching nuclides to ROIs is accomplished by using the centroid of a fitted peak and finding emission close in energy. The matching is weighted by the emission intensity so a strong emission will be chosen over a weak if they are the same distance from the peak centroid. By selecting nuclides the auto matching of nuclides to ROIs, including from using the button inside the *ROI Editor*, are limited emission from the selected nuclides. Using auto match nuclides against all possible candidates are often unreliable, especially if the energy calibration is imperfect.

The peak search is based on Savitzky-Golay smoothing and the filter derivatives to compute peak candidates. The computed candidates are filtered, the accepted peaks are then added as ROIs. The length of the window in the Savitzky-Golay filter can be set in units of channels.

2.2.6 Console

The console allows for entering text commands to control the application and running scripts for simple automation. See subsection 2.8 for information on available commands and their usage.

The Console can be used for light automation by using the `exec` command and providing a file path. The file is executed line by line and can used to quickly start set up a measurement session.

2.2.7 System Log

The System Log displays messages from the application and detector clients. If something crashes the error and traceback will be displayed here.

2.3 Gamma Tools

2.3.1 Calibration

Calibration is the operation of setting the energy each channel corresponds to.



Figure 2.17: Calibration window of a spectrum with 4 ROIs and assigned nuclides.

1. Which spectrum should be used for the calibration. For each calibration, data from only one spectrum is allowed.
2. Add a calibration point, that is not associated with a ROI. Measured and Reference energies must be set manually.
3. Current calibration points detected and their corresponding information. When unchecking a row it will be ignored in the calculation.
4. Polynomial degree of the calibration function and the current coefficients. Coefficients can be changed manually but will be updated by pressing `Calculate`.
5. The instrument used for acquiring the spectrum. This is fixed and can not be changed here, instead edit the spectrum before starting the calibration, see section **??**.
6. Plot showing the energy as a function of channels based on the latest calculated coefficients.
7. Assign the coefficients to the *Unique Instrument* shown in (5.).
8. Assign the new calibration coefficients and recalculate the x-axis for the spectrum selected in (1.), updating the view in the Spectrum-tab. Note, this will not change the calibration of an already started spectrogram.

2.3.2 Deconvolution

This tool does more than simply reducing the effect of energy resolution but will still be referred to as a deconvolution operation. Deconvolution in this sense is more similar to reconstruction as in image reconstruction in nuclear medicine, it employs the ML-EM algorithm. By using a detector response matrix, most simply acquired using Monte Carlo-simulations, the effects of the detector resolution, scattering in the detector and other effects can be reduced. The response matrix represents the expected detector response to a photon emitted at a certain energy, see Table 4.10 and section **??**

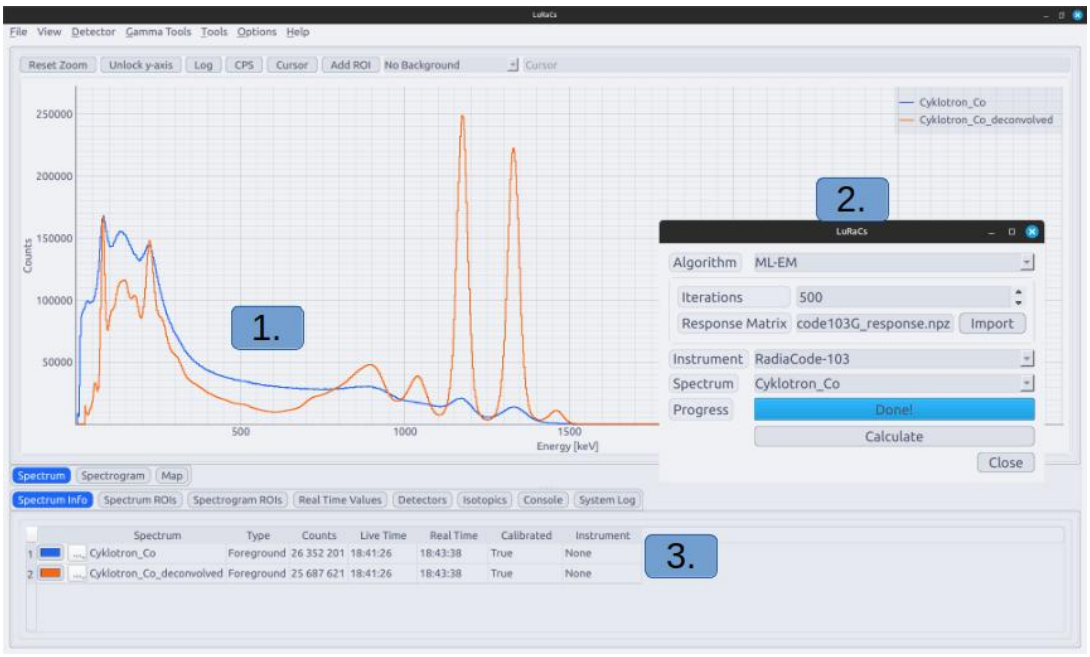


Figure 2.18: A Co-60 spectrum deconvolved using ML-EM and a response matrix for a Radiacode-103G detecting a parallel beam of monoenergetic photons.

1. The original Co-60 spectrum in blue and the deconvolved spectrum in orange.
2. Control window for the deconvolution operation, see Table 2.7.
3. The deconvolved spectrum is added as new spectrum, copying information from the original.

Table 2.7: Description of the deconvolution window.

Field	Description
Algorithm	Which algorithm should be used. Options are `ML-EM` (use this) and `Generalized Richardson-Lucy`.
Iterations	The number of iterations the algorithm should run.
(ML-EM) Response Matrix	The path to a .npz file containing the detector response matrix to be used.
Instrument	Instrument providing information for the algorithm. For instance, `Generalized Richardson-Lucy` is based on the resolution saved in the instrument.
Spectrum	Which spectrum should the operation be executed on.

2.3.3 Efficiency

The intrinsic efficiency is an important detector property and describes the fraction of photons hitting the detector that is actually detected.

2.3.4 Peak Features

The *Peak Features*-tool calculates the expected position of gamma spectrum features from a known photopeak energy. The features can then be marked with lines in the *Spectrum*-tab.

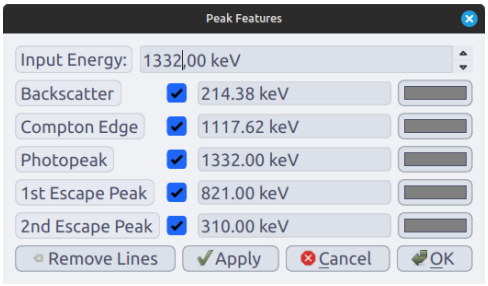


Figure 2.19: Peak Features tool with the 1332 keV energy of Co-60 set.

- `Remove Lines`: Removed and lines placed in the *Spectrum*-tab.
- `Apply`: Adds lines to the *Spectrum*-tab.
- `Cancel`: Closes the dialog with no other action taken.
- `Ok`: Applies the lines and closes the dialog.

2.3.5 Resolution

The energy resolution describes how well the detector can resolve detected photons of different energies. A detector with low resolution results in broad peaks which can cover smaller peaks, making them harder to detect.

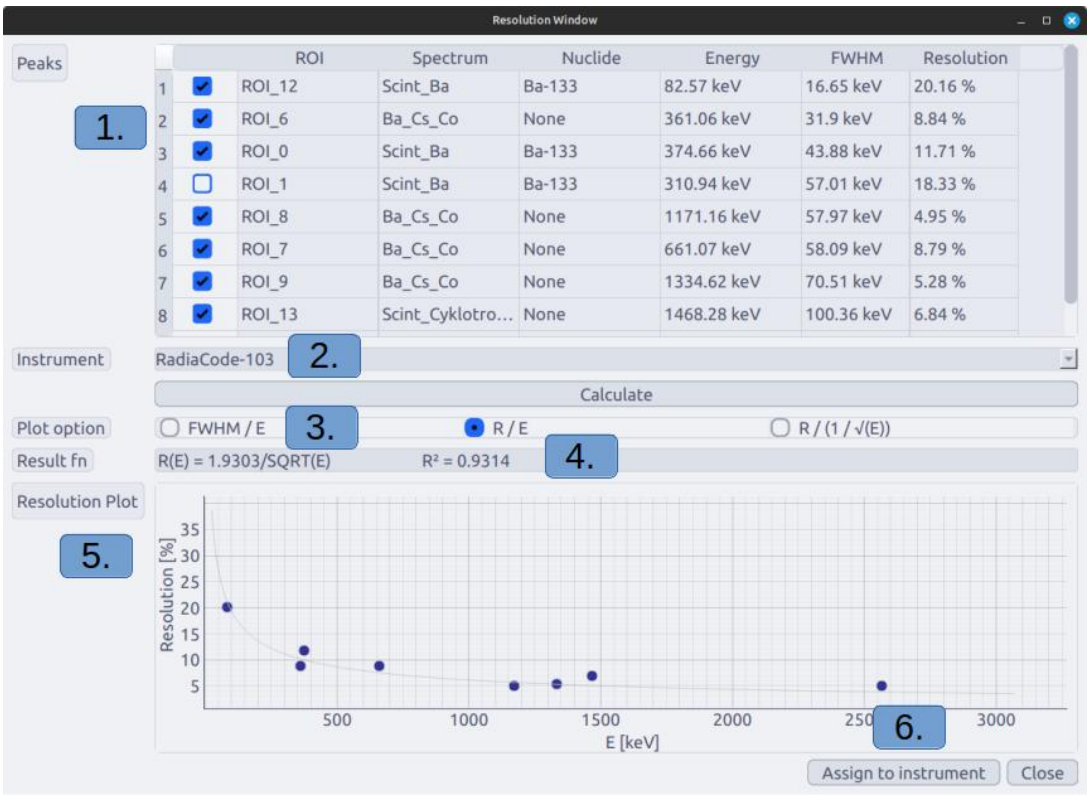


Figure 2.20: Resolution window with several different spectra measured with a 3"x3" NaI(Tl) scintillator loaded. Using `Tabbed Spectrum View` so the ROIs are only evaluating one spectrum each.

1. Fitted peaks from all loaded spectra and ROIs. A ROI without a fit can not be included. Unchecked entries will be ignored in the calculation.
2. Instrument to be given the calculation result. Can be Unique or Generic.
3. Options for how the calculation results should be plotted. These do not affect the results, only the view.
4. The resolution function, coefficients are in arbitrary units and not % (% = au * 100). The R^2 value of the fitted function compared with the data points.
5. Assign the calculation resolution calibration to the instrument set in (2.).

2.4 Detector Connection & Control

Connecting detectors using the GUI is facilitated through a dialog for each type of connection, they are found under `Detector -> Connect Bluetooth/USB/Network`. Connecting via Bluetooth or USB requires the application to scan for available devices. The serial number of the connected devices are then displayed, allowing the user to select the appropriate device. Connecting to network devices requires knowing the IP address and type of the device. Connecting to network detectors have the unique option to use the detector as a GPS. At the moment, this is primarily intended for the Ortec Detective X which has a built in GPS. When connecting to serial GPS devices the application scans serial ports and lists suggestions. The port can also be entered manually. The baud rate is typically 4800 but a selection of other common rates are also available.

Connecting the a network devices requires knowing the device IP and type beforehand. Connecting to network detectors have the unique option to use the detector as a GPS. This is primarily intended for the Ortec Detective X which has a built in GPS.

When connecting to serial GPS devices the application scans serial ports and lists sug-

gestions but the port can also be entered manually. The baud rate is typically 4800 but a selection of other common rates are also presented.

2.5 Data Library

The *Data Library* allows for saving data objects in the application it self for easy access. It also allows for limited editing of previously saved objects. It contains the following object types *Spectra*, *Spectrograms*, *ROIs*, *Unique Instruments* and *Generic Instruments*.

All data items saved in the Data Library can be exported to a single zip-file, with a specific internal structure (see **??**), using `File -> Export Data Library`. Such a file can be imported with `File -> Import Data Library`.

Spectrum Library

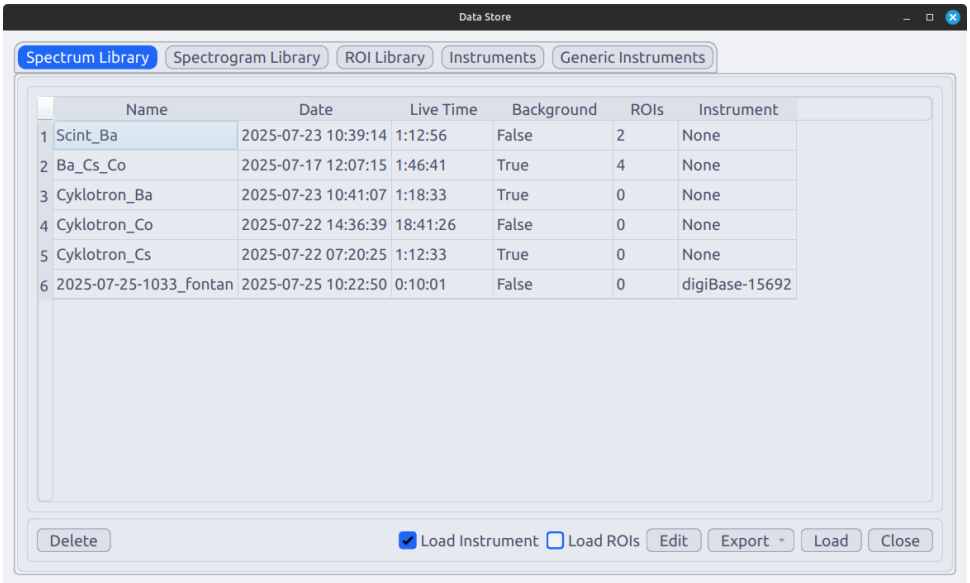


Figure 2.21

Table 2.8: Description of the possible operations in toolbar at the bottom of the Spectrum Library-tab

Field	Description
Delete	Remove selected items.
Load Instrument	Load the instrument contained in the specific spectrum file. If not selected LuRaCs will attempt to match the instrument name to an existing *Unique Instrument* save in the Data Library.
Load ROIs	Should the ROIs contained in the spectrum be loaded with the spectrum.
Edit	Edit the spectrum name or remark.
Export	Export the spectrum. Available formats: `XML`, `CSV` and `xlsx`.
Load	Load the spectrum into view.
Close	Close the data Library.

Table 2.9: Description of columns in the Spectrum Library-tab

Column	Description
Name	Spectrum name
Date	Date when the measurement of the foreground started
Live Time	Live time of the foreground
ROIs	Number of ROIs saved with the spectrum, if any.
Instrument	Which instrument is saved with the spectrum if any. If an instrument with the same name already exists in the Data Library, it will be used instead of the instrument saved with the spectrum. This can be changed in setting.

Spectrogram Library

All created spectrograms are placed in the Spectrogram Library. From here they can be loaded back for analysis or continued measurement.

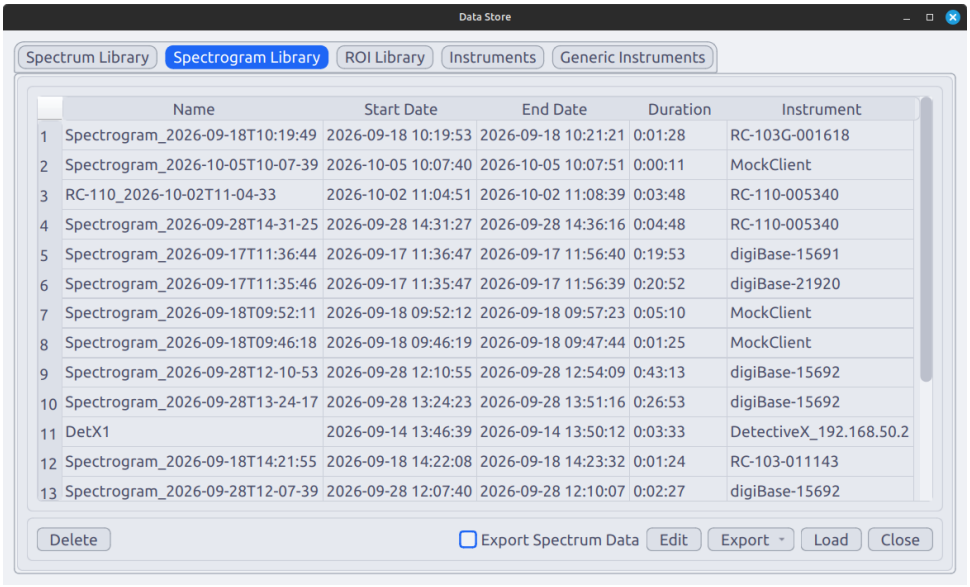


Figure 2.22

Table 2.10: Description of the possible operations in toolbar at the bottom of the Spectrogram Library-tab

Field	Description
Delete	Remove selected items.
Export Spectrum Data	If checked the data for individual channels are included when exporting, everything else is exported the same.
Edit	Edit the spectrogram name.
Export	Export the spectrogram to another file. Available formats: `.db` (SQLite3) and `xlsx`.
Load	Load the spectrum into view.
Close	Close the data Library.

Table 2.11: Description of columns in the Spectrogram Library-tab

Column	Description
Name	Spectrogram name
Start Date	Date when the measurement started.
End Date	Date when the measurement was last stopped.
Duration	Total measurement duration.
Instrument	Identifier for the detector used for acquiring the spectrogram.

ROI Library

The ROI Library allows for saving references to ROIs. These can later be loaded to a spectrum or spectrogram. This means the roi options, like the energy bounds, fitting options and assigned nuclide, are saved but data like fitting parameters and peak counts are not.

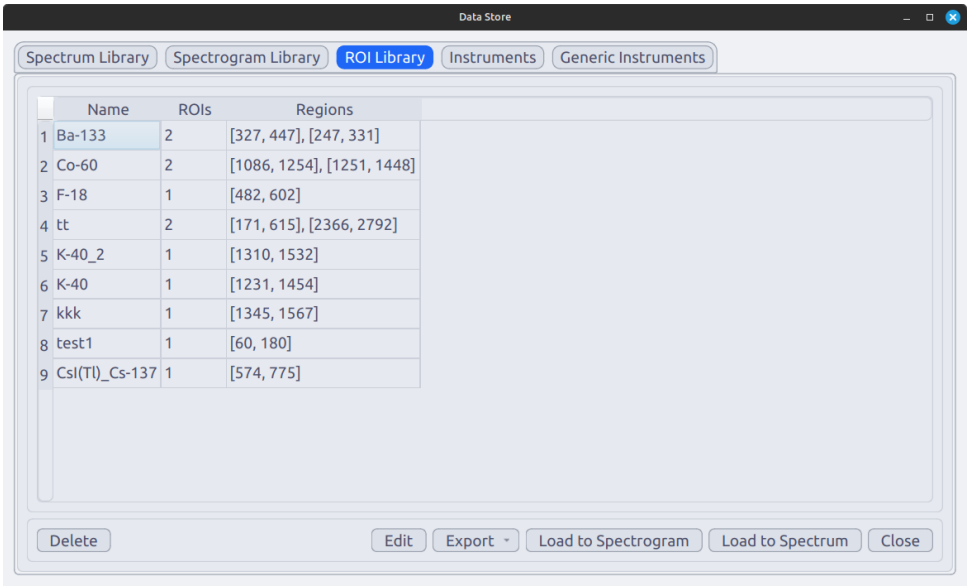


Figure 2.23

Table 2.12: Description of the possible operations in toolbar at the bottom of the ROI Library-tab

Field	Description
Delete	Remove selected items.
Edit	Edit the ROI collection name and the ROIs contained within, see Figure 2.24.
Export	Export the ROIs to another file. Available formats: `.xml`.
Load to Spectrogram	Load the ROIs as spectrogram ROIs.
Load to Spectrum	Load the ROIs as spectrum ROIs.
Close	Close the data Library.

Table 2.13: Description of columns in the ROI Library-tab

Column	Description
Name	Name of the ROI group.
ROIs	Number of ROIs in the group.
Regions	The bounds of all ROIs in the group.



Figure 2.24: ROI editor inside the Data Store. From the nuclide library, it is possible to change the ROI alias/name as well as upper and lower bounds, nuclide, emission.

Unique-/Generic Instruments

The tabs for the Unique and Generic Instruments are very similar and therefore described in the same section.

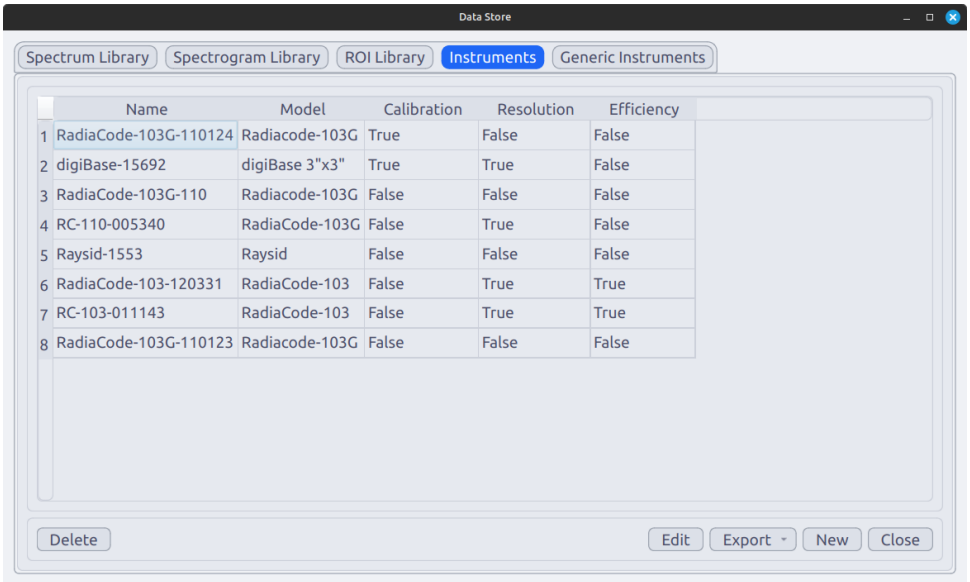


Figure 2.25: Instruments-tab in the Data Library, with eight listed Unique Instruments.

Table 2.14: Description of the possible operations in toolbar at the bottom of the Instruments-tab

Field	Description
Delete	Remove selected items.
Edit	Edit an existing instrument, see Figure 2.26.
Export	Export the instrument data to another file. Available formats: `.xml`.
New	Create a new instrument. *Unique Instruments* can inherit data from *Generic Instruments*.
Close	Close the data Library.

Table 2.15: Description of columns in the Instruments-tab

Column	Description
Name	Instrument name (Only for *Unique Instrument*)
Model	Name of the instrument model. Ex. RadiaCode-110 or Identifinder R400.
Calibration	Shows if the instrument stores an energy calibration. (Only for *Unique Instrument*)
Resolution	Shows if the instrument stores an energy resolution.
Efficiency	Shows if the instrument stores an intrinsic efficiency.

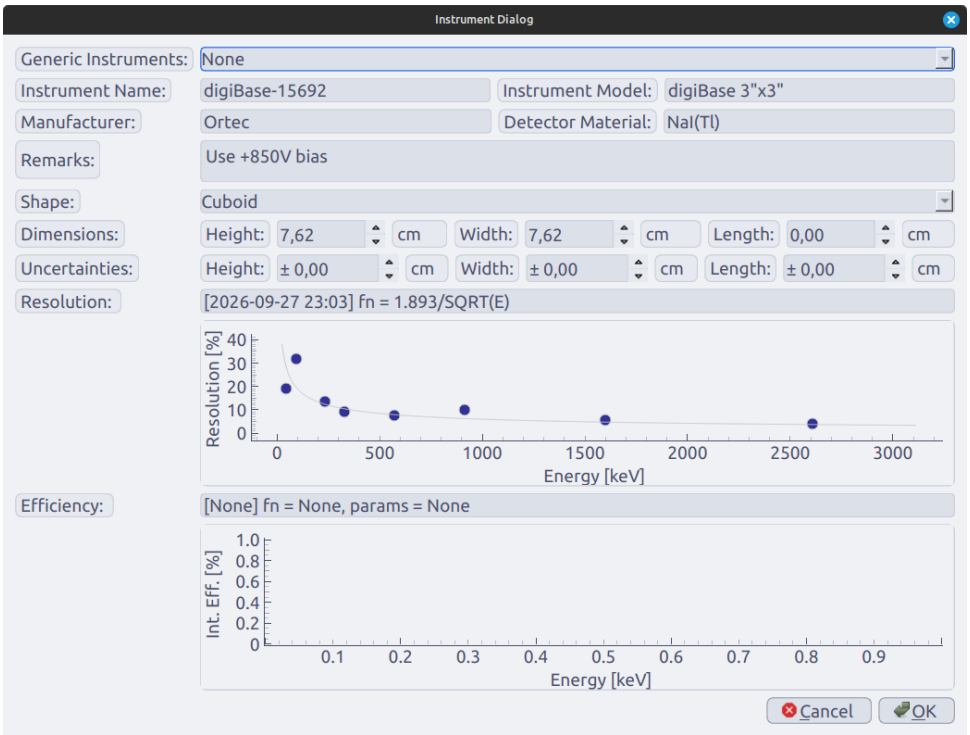


Figure 2.26: The dialog for creating a new instrument or editing an existing one. 'Instrument Name' can not be changed for a generic instrument.

2.6 Settings

The settings are divided into `Settings` that contain basic and cosmetic settings, and `Advanced Settings` containing settings that ordinary users rarely need to interact with.

Table 2.16: Application settings.

Setting	Description
Appearance	
Theme	Select the visual theme used by the application interface and plots.
Plot Colour Scheme	Select the color scheme used when automatically assigning colors to plotted spectra. Options are *MatplotLib*, `LibreOffice` and `Catppuccin`
Spectrum Draw Style	Select how spectra are rendered. Available options are *Pen*, *Brush*, and *Pen + Brush*.
Font Size	Set the application font size in points.
Spectrum Plot xMax	Set the maximum energy displayed on spectrum plots, in keV.
Verbose Calculation Logging	Enable additional logging during calculations. This can be useful when diagnosing calculation or processing problems.
Import Options	
Load ROIs on import	Automatically load regions of interest when importing a spectrum from a file.
Load instrument on import	Automatically load instrument information when importing a spectrum from a file.
Load calibration from instrument	Load the energy calibration associated with the instrument when importing data from the Data Store or from a file.

Table 2.17: Advanced application settings.

Setting	Description
Device Running	
Update Loop Delay	Set the delay between iterations of the device monitoring and update loop, in seconds.
Spectrum Update Delay	Set the delay between spectrum updates during live acquisition, in seconds.
Device Scanning	
GUI Scan Length	Set the duration of Bluetooth Low Energy scans initiated through the graphical user interface, in seconds.
Headless Scan Length	Set the duration of Bluetooth Low Energy scans when no the scan is not tracked in the GUI, in seconds.
Optimizer	
Optimizer Max Iter	Set the maximum number of iterations allowed by the optimizer.
Optimizer Tolerance	Set the convergence tolerance used by the optimizer.
Use Chi^2 Weight	Enable χ^2 weighting during optimization.
Logging	
Log Exceptions	Enable logging of unhandled exceptions.
Write logs to file	Enable writing log messages to the application log file.
Write logs to console	Enable writing log messages to the terminal or console.

Continued on next page

Table 2.17 – continued from previous page

Setting	Description
Headless Log Buffer Length	Set the maximum number of log entries retained in the in-memory log buffer. Changes to this setting require an application restart.
Spectrogram	
Spectrogram View Buffer	Set the number of recent spectrogram data points retained for display.
Maps	
Save extra GPS data	Enable saving additional GPS information with map-related data.

2.7 Command Line Arguments

Command Line Arguments can be provided when starting the application from a terminal.

Table 2.18: Command-line options available when starting LuRaCs.

Option	Description
General	
`-debug`	Launch the application in debug mode. A mock device is added automatically for testing and development.
`-headless`	Run the application without the graphical user interface, using terminal mode instead.
`-clear_local_cache`	Remove all locally stored files and exit the application.
Data Import	
`-import_spectrum <file> [file ...]`	Load one or more spectrum files when starting the application. Each specified file must exist and be supported by the application's file importer.
`-import_rois <file>`	Load regions of interest from the specified XML file.
`-roi <lower> <upper>`	Add a region of interest with the specified lower and upper energy bounds in keV. The option can be specified multiple times to add several ROIs.
`-nuclides <nuclide> [nuclide ...]`	Preset one or more nuclides in the isotopics interface. This option is applied when the graphical user interface is available.
Device Connections	
`-ble <device> [device ...]`	Attempt to establish Bluetooth Low Energy connections using the specified device names.
`-bluetooth <device> [device ...]`	Long-form alias for `-ble`. Attempts Bluetooth Low Energy connections using the specified device names.
`-usb <device> [device ...]`	Attempt to establish USB connections using the specified device names. Matching is performed against detected USB device product names.
`-nw <ip> [ip ...]`	Attempt to connect to network devices using their IP addresses.

Continued on next page

Table 2.18 – continued from previous page

Option	Description
`-network <ip> [ip ...]`	Long-form alias for `-nw`. Connects to network devices using their IP addresses.
`-serial_gps <interface>`	Connect a GPS receiver using a USB serial interface. The specified value identifies the serial interface used by the GPS receiver.
Maps	
`-map_url <URL>`	Load an online map from the specified URL at application startup. Only one map source may be specified at startup.
`-map_file <path>`	Load a local offline map from the specified `.pmtiles` file at application startup. Only one map source may be specified at startup.

2.8 Command Line Interface (CLI)

LuRaCs is primarily operated through the GUI but it also provides a terminal-based command line interface, allowing for controlling detectors without a GUI. This may be useful if remote measurements or measurements run on a Raspberry Pi is attached to a drone. This mode shares most of the same commands as the GUI Console with the exception of visualisation tools.

This mode is started by running LuRaCs in a terminal with the `-headless` flag.

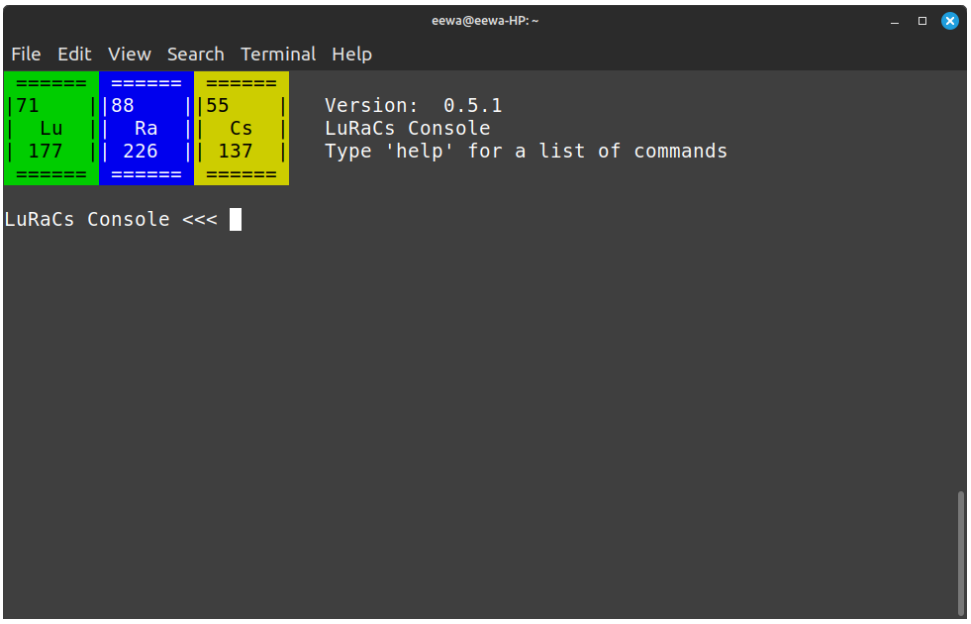


Figure 2.27: LuRaCs command line interface in terminal mode.

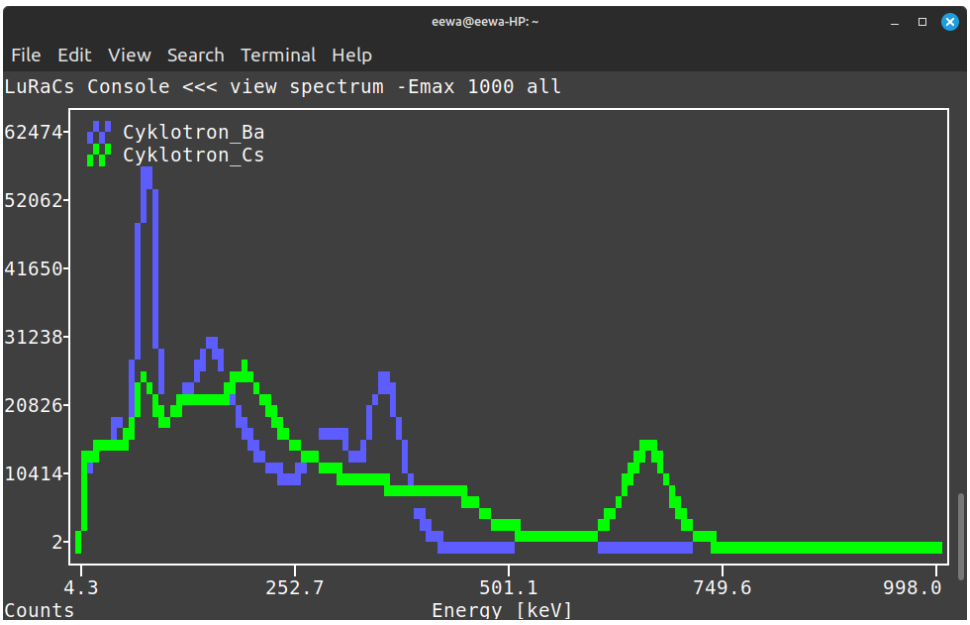


Figure 2.28: Two spectra viewed through the command line using the command `view spectrum -Emax 1000 all`. The `-Emax 1000` flag limits the x-axis to < 1000 keV.

Table 2.19: Command-line interface command reference.

Command	Description
General	
`help`	Display the available commands and their descriptions.
`clear`	Clear the console and return the application to start.
`exit, quit, shutdown`	Exit the application.
Device Management	
`device scan usb`	Scan for connected USB devices.
`device scan ble`	Scan for nearby Bluetooth Low Energy devices.
`device connect usb <device>`	Connect to a USB device matching the specified name.
`device connect ble <device1> [device2 ...]`	Connect to one or more Bluetooth Low Energy devices.
`device disconnect <device>`	Disconnect the specified device.
`device disconnect all`	Disconnect all connected devices.
Resource Listing	
`list devices`	Display all connected devices.
`list spectra`	Display all loaded spectra.
`list spectrogram`	Display all loaded spectrograms.
`list rois`	Display all loaded regions of interest.
Indexes	
`index spectrum`	Display indexed spectrum files.
`index spectrogram`	Display indexed spectrogram files.
`index roi`	Display indexed ROI files.
Viewing Data	

Continued on next page

Table 2.19 – continued from previous page

Command	Description
`view spectrum <name>`	Display the specified spectrum. The spectrum can be selected by name or by its index in the loaded spectrum list.
`view spectrum -n <name>`	Specify the spectrum to display by name.
`view spectrum all`	Display all loaded spectra on the same plot.
`view spectrum -A`	Display all loaded spectra on the same plot.
`view spectrum -Emin <energy>`	Set the lower energy limit of the displayed spectrum, in keV.
`view spectrum -Emax <energy>`	Set the upper energy limit of the displayed spectrum, in keV.
`view spectrum -cps`	Display the spectrum as counts per second (CPS) instead of total counts.
`view spectrum -log`	Display the spectrum using the logarithmic scaling.
`view spectrum -bkgSub`	Display the background-subtracted spectrum, if the spectrum has a background.
`view rois`	Display the currently loaded regions of interest.
`view log`	Display recent application log entries.
ROI Management	
`roi <lower> <upper>`	Add a region of interest between the specified lower and upper bounds.
`roi -c`	Clear all regions of interest.
Spectrogram Management	
`spectrogram start <device>`	Start a spectrogram for the specified device.
`spectrogram start all`	Start spectrograms for all connected devices.
`spectrogram pause <name>`	Pause the specified spectrogram.
`spectrogram pause all`	Pause all running spectrograms.
`spectrogram unpause <name>`	Resume the specified spectrogram.
`spectrogram unpause all`	Resume all paused spectrograms.
`spectrogram load <path>`	Load a spectrogram from the specified file.
`spectrogram unload <name>`	Unload the specified spectrogram.
`spectrogram unload all`	Unload all loaded spectrograms.
`-n <name>`	Specify a custom name when starting a spectrogram.
`-i <seconds>`	Set the spectrogram save interval. Default: 1 second.
`-c <count>`	Set the number of spectra to concatenate before saving. Default: 1.
Map Management	
`map url <https-URL>`	Load an online map from the specified HTTPS URL.
`map file <path>`	Load a local offline map from a PMTiles file.
Monitoring	
`watch`	Continuously display live information from connected devices. Press `Ctrl+C` to stop.
Automation	

Continued on next page

Table 2.19 – continued from previous page

Command	Description
`exec <script-file>`	Execute commands from a script file. Empty lines and lines beginning with `#` are ignored.

3 Examples

3.1 Quick Starting Several Radiacode Detectors for mobile measurements

This example demonstrates how the terminal can be used to effectively control several detectors at once, without having to repeat the connection and re-start the measurement procedure for each connection. For the example, three Radiacode detectors are used but the principle is the same for all other types of detectors. The code below illustrates how the commands start an offline map, connect to a GPS and the instruments, and start spectrograms for all detectors.

```
# Start the map first, cacheing a large map will require all available resources,
↪ that will cause the application to freeze momentarily. This can disturb the
↪ sampling of the detectors. map file /Downloads/sverige.pmtiles
map file ~/Downloads/sverige.pmtiles

# Once the map is loaded and connect to the GPS # This assumes that only one GPS
↪ device is connected to the PC. If several GPS are connected, the GPS with the
↪ first serial port is selected to be connected.
gps connect

device connect ble Radia
# This command will scan all Bluetooth devices and match the given name to the
↪ serial number of the found devices, device connect ble Radia # This will
↪ connect all detectors with 'radia' in the name, note that matching is case
↪ insensitive # Check if all detectors are connected. If not, repeat the above
↪ command list devices
list devices

# Start recording spectrograms for all devices
spectrogram start all

# When the measurements are concluded stop the recordings
spectrogram pause all
```

3.2 Connecting Detective X using direct ethernet

LuRaCs communicates with an Ortec Detective X detector using the *Mobile Field Kit* API described in the Detective X manual. Using this interface requires a network connection between the instrument and a host computer. A simple and stable way to establish such a connection is a direct ethernet cable from the instrument to the PC with no router in between. This method requires some setup before *LuRaCs* can connect to a Detective X and is described in this example.

The first step is to configure the ethernet interface on the Detective X. Go to `Settings` → `Communications` → `Ethernet`. Ensure that ethernet is enabled and disable DHCP. Manually enter the IP-address and network mask. A good choice of IP is **192.168.50.2**, as long as it does not interfere with another connected device, and set the network mask to **255.255.255.0**.

The second step is to configure the PC to have a static IP address on the ethernet connection. The IP-address will be **192.168.50.1** and the network mask 255.255.255.0, leave gateway empty. Examples are shown for Windows 11 (Figure 3.1) and Linux Mint (Figure 3.2) but the principle is the same for most systems.

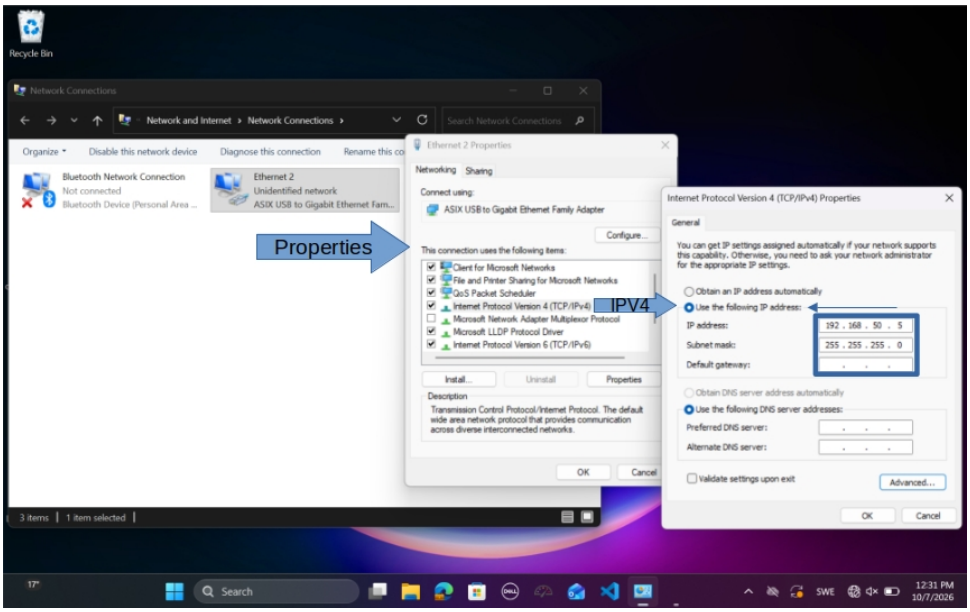


Figure 3.1: Steps for settings a static IPV4 address on Windows 11.

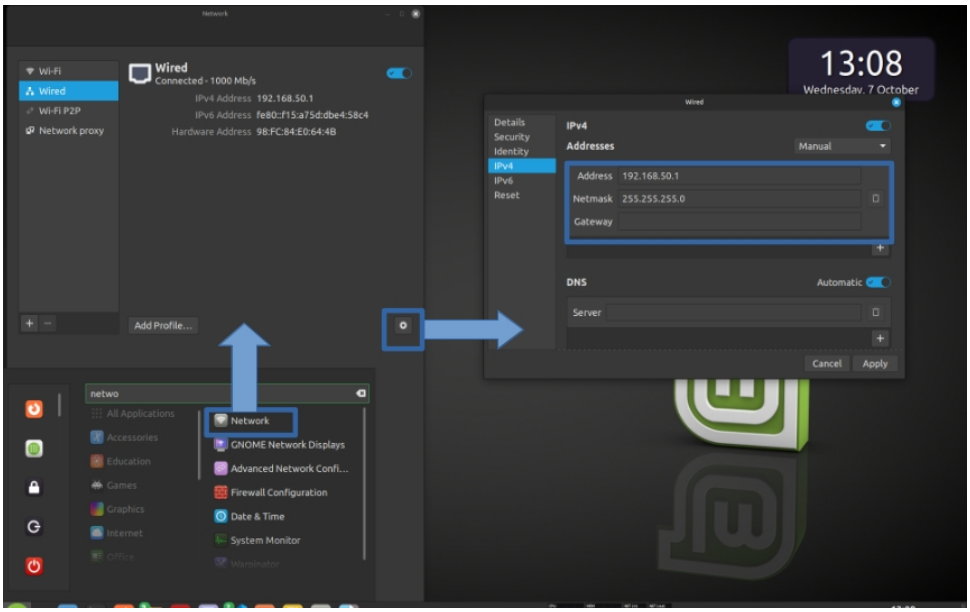


Figure 3.2: Steps for settings a static IPV4 address on Linux Mint 21.3 (Cinnamon).

Once the network is configured check the connection by pinging Detective X. The ping the same on most operating systems and in this example would be `ping 192.168.50.2`. On most Linux systems, if the ping fails try restarting the network manager with the following command `sudo systemctl restart NetworkManager`. If that does not work, try rebooting the PC.

If the ping succeeds everything is ready. In *LuRaCs* go to `Devices` → `Connect Network` and enter the instrument IP, 192.168.50.2 in this example.

3.3 Preparing and Running the LU mobile detector setup

This setup consists of two 4 L NaI(Tl) scintillation detectors with digiBase MCAs and one HPGe detector with a digiDart system, all mounted in a car, equipped with an external usb connected GPS.

1. Ensure that the digiBase firmware is imported. `File -> Drivers`. If not se section 1.3
2. Connect the detectors, neither the digiBase nor the digiDart save their calibration when used with LuRaCs. Hence, only the channel numbers will be shown on the energy axis. When connecting from a Linux system, serial numbers and detector type will be displayed for easy identification. On a Windows system they must be identified through the Windows Device Manager. `Detector -> Connect USB`
3. Enable the correct voltage on each detector. This information might be remembered by the detector MCA.. Otherwise check the serial numbers of the detectors to find which is which. `Detector-tab -> 'A detector Row' -> '...'-button -> Settings -> MCA-tab -> HV: Enable`
4. (Optional) Calibrate the detectors with the K-40 and Tl-208 peaks and the 0 channel using the Calibration tool. Apply to the spectrum. `Gamma Tools -> Calibration`
5. With the calibration a new instrument can now be created from the Detectors-tab to save the calibration. It will automatically be fetched the next time the detector is connected. `Detectors-tab -> 'A detector Row' -> '...'-button -> Make Instrument -> 'Save Calibration' is checked`
6. Start the map. If an offline map is used, start it before any spectrogram, loading a large map might interfere with sampling of the instruments data, that disrupts the even save intervals. `Map-tab -> Map Options -> Load Offline Map`
7. Connect the GPS. `Detector -> Connect Serial GPS`, determine the correct port if required and connect. LuRaCs will display all available serial ports and if only one device is connected it is trivial to identify it. 4800 is likely the correct Baud Rate.
8. Check that the GPS has a signal. `Map-tab -> Line Bellow the map -> 'Fix' should be 'Yes'` It might take some time for the GPS to warm up.
9. Ensure all detectors are running by clearing all spectra. `Detector -> Clear All Spectra`
10. Start the spectrograms individually, `Spectrum-tab -> Start New` or use the console, `Console -> 'spectrogram start all'`.

4 Internal Function

4.1 Code Structure

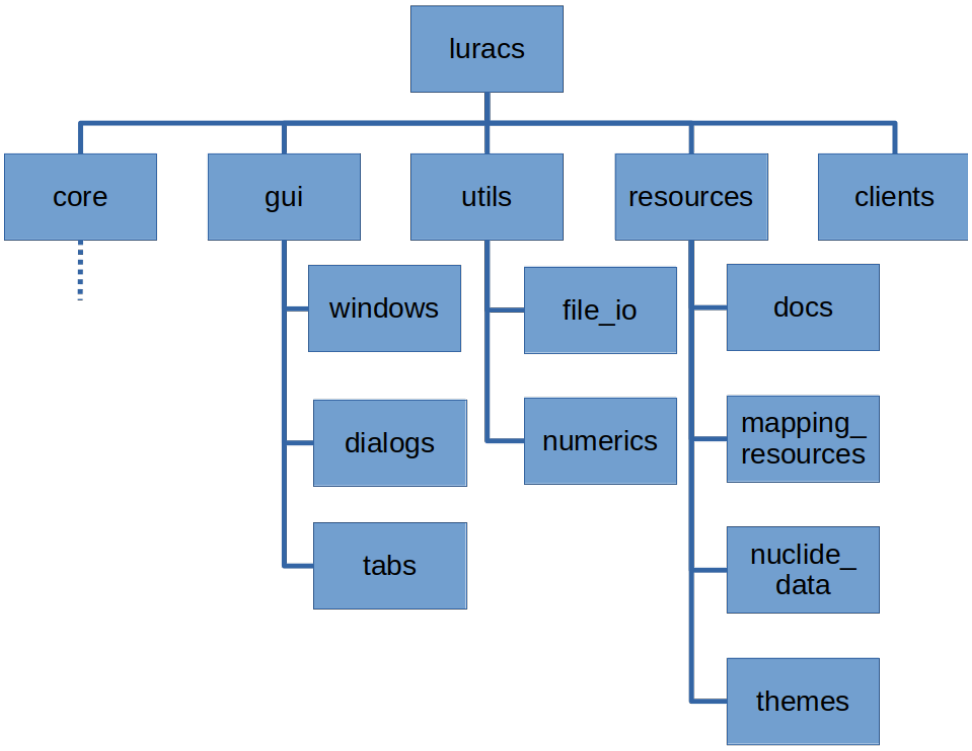


Figure 4.1: Flowchart illustrating the structure of the LuRaCs code

core

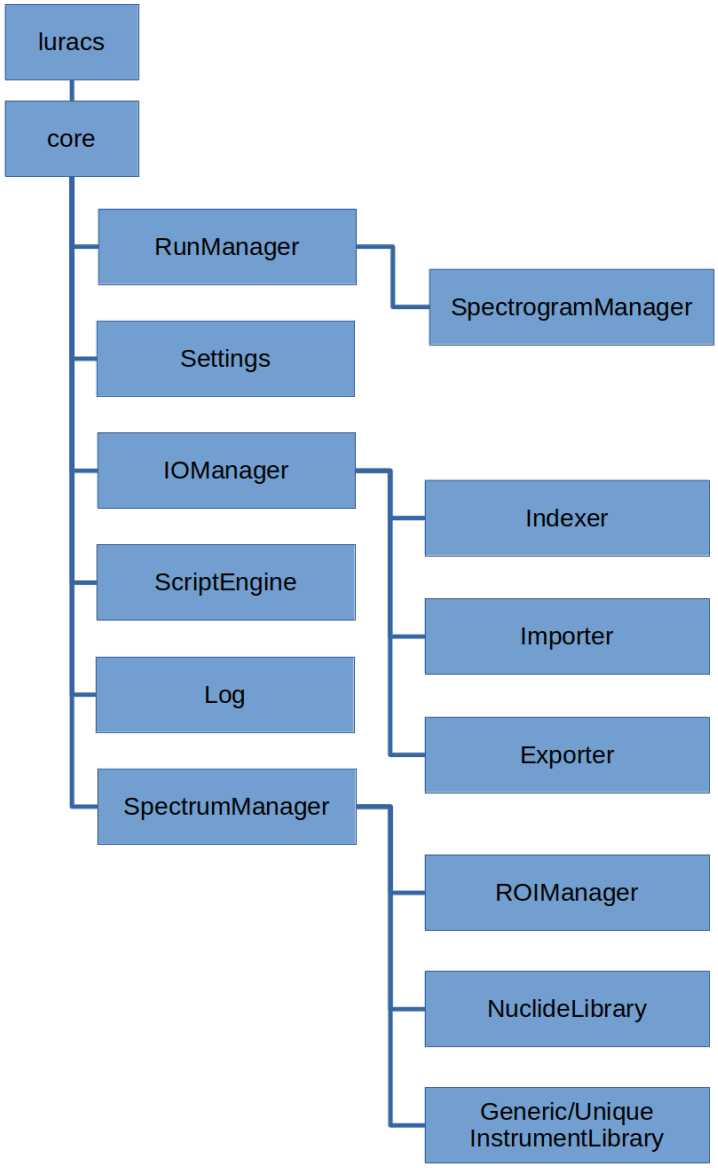


Figure 4.2: Flowchart of the structure of the LuRaCs core module

The core of the application is built on a number of singleton classes that manage the global state of the program. These are reachable from anywhere in the program.

Settings The settings contain modifiable parameters that affect the running of the program. It is divided in the the following parts. Settings can also be used for communication within the program as when a setting is changed using the `update_setting` function a signal is emitted from `Settings` with the group of the setting, name of the setting and the new value.

- **Appearance** contain simple settings mostly controlling appearance of the GUI and preferences for data import.
- **Paths** contain file and system paths to resources and stored data.
- **Temp** contain values that are reset upon shutdown.

- **State** saves mostly last used values for convenience and is capable of saving and loading complex data types like dequeues.
- **Advanced** contains advanced options that most users rarely need to change.

RunManager The RunManager handles connecting and sampling instruments. It owns its own thread running an async event loop, this to avoid instrument sampling being affected by events in the GUI thread and vice versa. Threadsafe communications are handled using Qt signals. The RunManager also owns the **SpectrogramManager** which tracks spectrogram and manages spectrogram ROIs.

ScriptEngine The ScriptEngine is responsible for converting text commands into actions. Like the RunManager it owns it own thread and communications are made using Qt signals. Available commands are described in the section on the CLI.

Log The Log exports a reference to the python logger "Application".

IOManager The IOManager provides functions for import and export of various files and manages indexes of internally stored file for the Data Store.

SpectrumManager The SpectrumManager is one of the most critical parts of the application. It owns all spectra, both from files and devices. Modifications of spectra are handles through it. It also owns the following components.

- **NuclideLibrary** contains loaded nuclide data and some functions for interacting with it.
- **ROIManager** handles spectrum ROIs and updates peak fits.
- **GenericInsturmentLibrary** contains generic instruments based on instrument models that can be used to store characterisc data and be used as basis for UniqueInstruments.
- **UniqueInsturmentLibrary** contains unique instruments that can be assigned to spectra. They differ from GenericInstruments in that they store information unique to each device like energy calibration.

The core also contains the helper class **Calculator** that is designed to easily assign a CPU intensive task to a separate thread and signal back the result using a Qt signals.

4.2 Wrapping Detectors

The implementation of detectors is written be extensible and make it easy to implement new instruments. This is facilitated though inheritance from an abstract class `DeviceWrapper`. This wrapping hides the individual device implementations and LuRaCs only interact through this wrapper interface. Creating a subclass from `DeviceWrapper` automatically registers the new wrapper for use as a detector connection.

4.3 Algorithms

4.3.1 Peak Fitting

Peak fitting are based in ROIs in which a peak is assumed to be. Each ROI can contain one peak but ROIs can merge so several peaks can share the same background estimation. One a ROI is defined the fitting process is as follows.

1. Determine a number of channels starting at the ROI and counting inwards on which to base the background.
2. A polynomial is fitted using the edge channels and their corresponding counts.
3. The polynomial is evaluated for every channel in the ROI or the merged ROI region and the result subtracted from the respective channels.
4. One or the sum of several Gauassian functions are fitted to the remainder.
5. Peak characteristics are extracted from the fitted paramters.

The uncertainties of the peak fitting is calculated as follows. Firstly, the counts in channel i, N_i is assumed to follow Poisson distribution, thus $\sigma_i = \sqrt{N_i}$. The uncertainty of each channel is used as weights for the polynomial fit.

4.3.2 Peak Search

4.3.3 Deconvolution

The deconvolution method implemented is based on the *Maximum Likelihood - Expectation Maximisation* or ML-EM algorithm.

Before reconstruction the response matrices are processed and rows for the requested values are interpolated. The method for interpolation is described by Jandel et al.[2].

4.4 File Types

4.4.1 Spectrum, ROI and Instrument (.xml)

Spectra are stored in the XML format and is interoperable with the ANSI/IEEE N42.42 format[3].

ROIs and Instrument data are stored as an extension to the N42 standard using the *LRC* namespace. All datatypes are converted to strings before XML serialisation. The tables bellow describe the XML fields and their corresponding Python type.

[2]M Jandel et al. "Decomposition of Continuum y-Ray Spectra Using Synthesized Response Matrix". In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 516.1 (Jan. 1, 2004), pp. 172–183. ISSN: 0168-9002. DOI: 10.1016/j.nima.2003.07.047.

[3]"Documentary Standards: ANSI/IEEE N42.42 Standard". In: *NIST* (). URL: https://www.nist.gov/programs-projects/documentary-standards-ansiieee-n4242-standard (visited on 10/06/2026).

Table 4.1: Fields stored in the XML representation of spectrum data.

XML field	Python type	Description
`RadMeasurement/@id`	`str`	Identifier of the radiation measurement.
`MeasurementClassCode`	`str`	Specifies whether the measurement is a foreground or background spectrum.
`StartDateTime`	`str`	Measurement start time formatted as an ISO 8601 date-time string.
`RealTimeDuration`	`str`	Real measurement duration encoded as an ISO 8601 duration.
`Spectrum/@energyCalibrationReference`	`str`	Reference to the energy calibration used by the spectrum.
`Spectrum/@id`	`str`	Identifier of the spectrum.
`LiveTimeDuration`	`str`	Measurement live time encoded as an ISO 8601 duration.
`ChannelData`	`np.ndarray`	Spectrum channel counts, converted to integer values before serialization.

Table 4.2: Fields stored in the XML representation of a ROI.

XML field	Python type	Description
`Roi/@id`	`str`	Internal tag identifying the ROI.
`Roi/@alias`	`str`	User-defined name or alias of the ROI.
`Roi/@spectrum_ref`	`str`	Name of the spectrum associated with the ROI.
`Roi/@version`	`str`	Version of the XML representation.
`PeakContinuum/@merge`	`bool`	Indicates whether the ROI is merged with other ROIs.
`PeakContinuum/@movable`	`bool`	Indicates whether the ROI can be moved interactively.
`PeakContinuum/@background_subtracted`	`bool`	Indicates whether background subtraction has been applied.
`PeakContinuum/RegionCounts`	`float`	Total counts within the ROI.
`PeakContinuum/LiveTime`	`float`	Measurement live time.
`PeakContinuum/LowerEnergy`	`float`	Lower energy boundary of the ROI.
`PeakContinuum/UpperEnergy`	`float`	Upper energy boundary of the ROI.
`Fit`	`str`	Type of peak fitting method applied to the ROI.
`Fit/@chi2_weighted_err`	`bool`	Indicates whether weighted χ^2 errors are used.
`BackgroundType`	`str`	Type of background model used for fitting.
`Peak/BackgroundParameters`	`np.ndarray`	Parameters of the fitted background model.
`Peak/FitParameters`	`np.ndarray`	Optimized parameters of the peak-fitting function.
`Peak/FitParametersErr`	`np.ndarray`	Uncertainties of the fitted peak parameters.
`Peak/Centroid`	`list`	Fitted centroid and its uncertainty.
`Peak/StandardDeviation`	`list`	Fitted standard deviation and its uncertainty.
`Peak/Amplitude`	`list`	Fitted amplitude and its uncertainty.
`Peak/PeakArea`	`list`	Integrated peak area and its uncertainty.
`Peak/NetCounts`	`list`	Net peak counts and their uncertainty.
`Peak/BkgCounts`	`list`	Estimated background counts and their uncertainty.
`Peak/GrossCounts`	`list`	Gross peak counts and their uncertainty.
`Nuclide/Name`	`str`	Name of the associated nuclide.
`Nuclide/Energy`	`list`	Emission energy and its uncertainty in keV.
`Nuclide/Intensity`	`list`	Emission intensity and its uncertainty in percent.
`Nuclide/Type`	`str`	Type of nuclear emission. *g* or *x-ray*.
`Nuclide/EmissionOrigin`	`str`	Origin of the nuclear emission information.

Table 4.3: Basic instrument information stored in XML.

XML field	Python type	Description
`Instrument/@generic_instrument`	`bool`	Indicates whether the instrument is generic rather than unique.
`Instrument/Name`	`str`	Name of the instrument.
`Instrument/Model`	`str`	Model of the instrument.
`Instrument/Manufacturer`	`str`	Manufacturer of the instrument.
`Instrument/DetectorMaterial`	`str`	Material of the detector.
`Instrument/DetectorShape`	`str`	Shape of the detector.
`Instrument/DetectorType`	`str`	Type of detector.
`Instrument/DetectorDimensions`	`tuple`	Physical dimensions of the detector in centimetres.
`Instrument/DetectorDimensionsUncertainty`	`tuple`	Uncertainties of the detector dimensions in centimetres.
`Instrument/Remark`	`str`	Optional free-text remark associated with the instrument.

Table 4.4: Energy resolution information stored in XML.

XML field	Python type	Description
`Resolution/@type`	`str`	Type of resolution model, set to `energy`.
`Resolution/CreatedTS`	`datetime`	Date and time at which the resolution model was created.
`Resolution/Function`	`str`	Function used to describe the energy resolution.
`Resolution/Parameters`	`list`	Parameters of the energy resolution function.
`Resolution/ResolutionPoints/@energy_unit`	`str`	Unit of energy for the resolution points, specified as keV.
`Resolution/ResolutionPoints/@FWHM_unit`	`str`	Unit of FWHM, specified as keV.
`Resolution/ResolutionPoints/DataPoint/FWHM`	`float`	Measured full width at half maximum.
`Resolution/ResolutionPoints/DataPoint/FWHMUncertainty`	`float`	Uncertainty of the measured FWHM.
`Resolution/ResolutionPoints/DataPoint/Energy`	`float`	Energy corresponding to the measured resolution.

Table 4.5: Intrinsic efficiency information stored in XML.

XML field	Python type	Description
`Efficiency/@type`	`str`	Type of efficiency model, set to `intrinsic`.
`Efficiency/CreatedTS`	`datetime`	Date and time at which the efficiency model was created.
`Efficiency/Function`	`str`	Function used to describe the intrinsic efficiency.
`Efficiency/Parameters`	`list`	Parameters of the intrinsic efficiency function.
`Efficiency/ ParametersUncertainty`	`list`	Uncertainties of the intrinsic efficiency parameters.
`Efficiency/ EfficiencyPoints/@energy_ unit`	`str`	Unit of energy for the efficiency points, specified as keV.
`Efficiency/ EfficiencyPoints/ @efficiency_unit`	`str`	Unit of efficiency, specified as unitless.
`Efficiency/ EfficiencyPoints/ DataPoint/Efficiency`	`float`	Measured intrinsic efficiency.
`Efficiency/ EfficiencyPoints/ DataPoint/ EfficiencyUncertainty`	`float`	Uncertainty of the measured efficiency.
`Efficiency/ EfficiencyPoints/ DataPoint/Energy`	`float`	Energy corresponding to the measured efficiency.
`Efficiency/Description`	`str`	Description of the intrinsic efficiency data or model.

Table 4.6: Energy calibration information stored in XML.

XML field	Python type	Description
`Calibration/ PolynomialOrder`	`int`	Order of the polynomial used for energy calibration.
`Calibration/Coefficients`	`list`	Polynomial coefficients used for channel-to-energy conversion.
`Calibration/ CalibrationPoints/ @energy_unit`	`str`	Unit of calibration energy, specified as keV.
`Calibration/ CalibrationPoints/ DataPoint/Energy`	`float`	Known energy of a calibration point.
`Calibration/ CalibrationPoints/ DataPoint/Channel`	`float`	Channel corresponding to the calibration energy.
`Calibration/CreatedTS`	`datetime`	Date and time at which the energy calibration was created.

4.4.2 Spectrogram (.db)

Spectrograms are saved to an SQLite3 database file. One file contains one spectrogram. Saved files can be loaded and measurement resumed. A spectrogram database contains three tables described in Table 4.7, 4.8 and 4.9.

Table 4.7: Schema of the `header` table.

Column	Type	Description
`id`	INTEGER	Identifier of the header record.
`created`	INTEGER	Creation time as a Unix timestamp.
`device_id`	TEXT	Identifier of the device producing the data.
`channels`	INTEGER	Number of spectrum channels.
`calibration`	TEXT	Calibration coefficients as a json list.
`concat`	INTEGER	Indicates whether the channels have been concatenated by a factor of 1 (not concatinated), 2, 4 or 8.
`save_interval`	REAL	Interval between data saves.
`meta`	TEXT	Additional metadata as json.

Table 4.8: Schema of the `summary` table.

Column	Type	Description
`id`	INTEGER	Identifier of the summary record.
`accumulated_spectrum`	BLOB	Accumulated spectrum data, zlib compressed array.
`total_duration`	REAL	Total duration of the recording in [s].
`total_dose`	REAL	Total accumulated dose in [µSv].
`last_update`	INTEGER	Timestamp of the last update as a Unix timestamp.

Table 4.9: Schema of the `spectrogram` table.

Column	Type	Description
`id`	INTEGER	Identifier of the spectrogram record.
`timestamp`	REAL	Timestamp of the measurement as a Unix timestamp.
`avg_cps`	INTEGER	Average counts per second multiplied by 1000.
`avg_dr`	INTEGER	Average dose rate as [µSv/s*1000].
`temperature`	INTEGER	Temperature during the measurement in [°C*1000].
`latitude`	REAL	Latitude of the measurement location.
`longitude`	REAL	Longitude of the measurement location.
`spectrum`	BLOB	Recorded spectrum during interval, zlib compressed array.
`meta`	BLOB	Additional metadata as a json string encoded with utf-8 and zlib compressed.

4.4.3 Detector Response Matrix (.npz)

Detector response matrices are stored as serialized and compressed numpy arrays (.npz) with pickle disabled. Each files contains three arrays described in Table 4.10.

Table 4.10: Contents of a response matrix .npz file.

Key	dtype	shape	Description
`response_matrix`	float32	2D (m x n)	Detector response values. First index is the simulated photon energy and second index is the normalized count bins (#bin / #fired photons)
`indices`	float32	1D (m)	Energy of the simulated monoenergetic photons of the corresponding row in `response_matrix` in keV
`bin_centres`	float32	1D (n)	Energy of the bin centres of the detector energy response in keV.

4.4.4 Simple map tracks (.geojson)

Spectrograms are the primary way to store GPS position data these are however not directly compatible with any other software without custom scripts. A spectrogram can be converted to the *GeoJSON* format, with the loss of some information, for compatibility and easier sharing. The *GeoJSON* format is generally compatible with dedicated GIS software, like *QGis*, for viewing measurements outside the embedded map.

Table 4.11: Collection-level contents of the GeoJSON mapping data file.

Key	dtype	Description
`type`	string	GeoJSON object type. Always set to `"FeatureCollection"`.
`name`	string	Name or title of the mapping dataset.
`metadata`	object	Metadata describing the mapping session, including device information, acquisition times, and additional metadata.
`metadata.device_id`	string	Identifier of the device used to acquire the measurements.
`metadata.start`	string	ISO 8601 timestamp indicating the start of the measurement period.
`metadata.end`	string or null	ISO 8601 timestamp indicating the end of the measurement period. Set to `null` if unavailable.
`features`	array	Collection of georeferenced measurement points.

Table 4.12: Contents of individual measurement features in the GeoJSON mapping data. Accessed as `features.xxx`

Key	dtype	Description
`type`	string	GeoJSON feature type. Always set to `"Feature"`.
`geometry`	object	Geographic representation of the measurement. The geometry is a GeoJSON `Point`.
`geometry.type`	string	Geometry type. Always set to `"Point"`.
`geometry.coordinates`	float	Longitude and latitude of the measurement location, given in GeoJSON order as `[longitude, latitude]`.
`properties.timestamp`	string	ISO 8601 timestamp indicating when the measurement was recorded.
`properties.count_rate`	float	Measured count rate at the corresponding geographic location.
`properties.dose_rate`	float	Measured dose rate at the corresponding geographic location.
`properties.other_data_points`	on	Additional measurements points, typically produced by spectrogram ROIs, saved as a JSON-object.

4.5 Embedded map

The embedded map is rendered using *MapLibre GL JS*[4] in a QWebEngine widget. The packages for MapLibre is included in the LuRaCs code and does not require any installation. MapLibre handles requesting tiles is connected to an online API it requires minimal

[4] *Maplibre/Maplibre-Gl-Js*. MapLibre, Sept. 21, 2026. URL: `https://github.com/maplibre/maplibre-gl-js` (visited on 09/21/2026).

work. Using maps offline works by using an embedded tile server run using *uvicorn* and *fastapi* to supply MapLibre with tiles. The embedded tile server runs on host 127.0.0.1 and port 8080. PMTiles are loaded using a fork of the *pmtiles* Python package, having improved cacheing and performance, incorporated in the LuRaCs code.

The code is written so that the the Python side handles most of the logic, using only simple calls to JS for creating or updating points, etc. To minimise the overhead incurred from the *Python* $\leftrightarrow$ *JS*-bridge send lists as a one object and let JS iterate over it, as opposed to sending every point over the bridge.

References

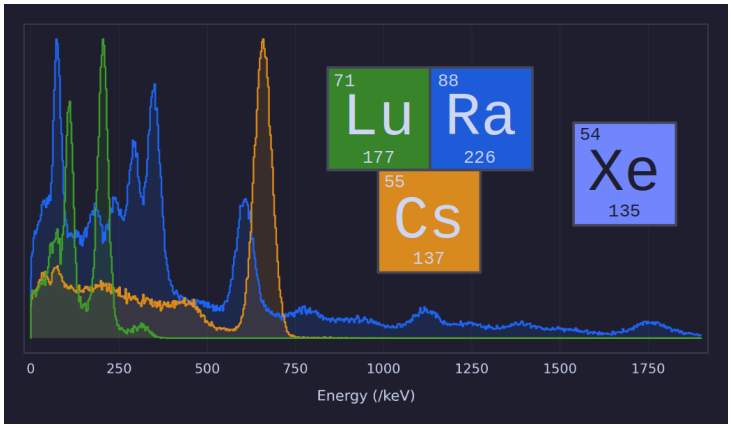
"Documentary Standards: ANSI/IEEE N42.42 Standard". In: *NIST* (). URL: https://www.nist.gov/programs-projects/documentary-standards-ansiieee-n4242-standard (visited on 10/06/2026).

Jandel, M et al. "Decomposition of Continuum y-Ray Spectra Using Synthesized Response Matrix". In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 516.1 (Jan. 1, 2004), pp. 172–183. ISSN: 0168-9002. DOI: 10.1016/j.nima.2003.07.047.

Maplibre/Maplibre-Gl-Js. MapLibre, Sept. 21, 2026. URL: https://github.com/maplibre/maplibre-gl-js (visited on 09/21/2026).

Sandialabs/InterSpec. Sandia National Laboratories, Aug. 28, 2025. URL: https://github.com/sandialabs/InterSpec (visited on 08/29/2025).

A LuRaCs-Xe



LuRaCs-Xe is a supplementary resource repository developed to accompany the main LuRaCs application. It provides a collection of example datasets, reference materials, and other resources intended to support the use, testing, and further development of LuRaCs. This resource is continually updated independently of the main application. Each component contains a README-file that provides information about the specific resource.

By bringing these resources together in a common repository, LuRaCs-Xe aims to facilitate reproducible analysis, provide practical examples for new users, and support the development and evaluation of analytical methods.

It currently contains the following sections:

- `example_spectra`: Spectrum files from measurements we have conducted with a verity of sources and detectors.
- `roi_references`: Predefined ROIs for common nuclides for use with LuRaCs. Especially useful when using spectrogram ROIs as the bounds have been determined from peaks with good statistics.
- `response_matrices`: Simulated detector response matrices for use in the spectrum deconvolution tool in LuRaCs.

B Change Log